MCTS-based Logic Restructuring for Timing Closure in Physical Synthesis

An M.Tech Project Report Submitted
in Partial Fulfillment of the Requirements
for the Degree of

Master of Technology

by

Naman Baveja

(244101029)

under the guidance of

Dr. Sukanta Bhattacharjee



to the

COMPUTER SCIENCE AND ENGINEERING PROGRAMME

INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI
GUWAHATI - 781039, ASSAM

CERTIFICATE

*This is to certify that the work contained in this thesis entitled "**MCTS-based Logic Restructuring for Timing Closure in Physical Synthesis**" is a bonafide work of **Naman Baveja** (**Roll No. 244101029**), carried out in the Computer Science and Engineering programme, Indian Institute of Technology Guwahati under my supervision and that it has not been submitted elsewhere for a degree.*

Supervisor: **Dr. Sukanta Bhattacharjee**

Designation: Assistant Professor

Month, of June 2026

Department of Computer Science and Engineering,

IIT Guwahati,

Assam

Acknowledgements

In the first place, I would like to express my sincere gratitude and heartfelt thanks to my thesis supervisor, **Dr. Sukanta Bhattacharjee**, Department of Computer Science and Engineering, IIT Guwahati, for his constant guidance, encouragement, and valuable insights throughout this project. His support not only helped me complete this work but also taught me many important lessons that will guide me in my future career.

I am also deeply thankful to **Dr. Chandan Karfa** for sharing his knowledge and providing invaluable guidance and support.

Furthermore, I am truly grateful to my parents and friends for their constant encouragement, emotional support, and belief in me. They have always stood by me and have been a pillar of strength.

Finally, I once again extend my deepest thanks to Prof. Sukanta Bhattacharjee, without whom this project would not have been possible.

Abstract

Timing optimization techniques in electronic design automation (EDA), such as gate sizing, are traditionally constrained by operating on a fixed netlist topology. Furthermore, because early logic synthesis lacks accurate physical data, its unreliable estimations for delay and area severely restrict Performance, Power, and Area (PPA) optimization. To overcome these bottlenecks, this thesis presents a physically-aware resynthesis framework that moves beyond fixed topologies to dynamically restructure logic. We introduce a timing-cone extraction and optimization engine that leverages real-time placement coordinates and wire parasitics to restructure critical paths. Guided by a Monte Carlo Tree Search (MCTS) strategy, this core logic transformation directly targets timing-violating endpoints to improve Total Negative Slack (TNS). Validated within the OpenROAD flow, the framework ensures clean netlist stitch-back and precise timing recovery, providing a highly efficient approach to post-placement and routing timing closure.

Contents

List of Figures **xi**

List of Tables **1**

1 Introduction **2**

1.1 Electronic Design Automation Overview 2

1.1.1 Front-End Design 3

1.1.2 Back-End Design 4

1.1.3 Sign-Off and Verification 4

1.2 Introduction to OpenROAD 5

1.3 Workflow of OpenROAD [1] 5

1.3.1 The Main Stages of Physical Design 6

1.3.2 Integrated Core Modules 7

1.3.3 Support Tools 8

1.3.4 Visualization and Control 10

1.4 Timing Optimization 10

1.4.1 Techniques in use for Timing Optimization [2] 11

1.4.2 Gate Resizing 11

1.4.3 Limitations of Current Timing Repair Methodologies 13

1.5 Contribution 14

1.6 Report Organisation 15

2 Reveiw of Prior Works **17**

2.1 The Scope and Limits of Electrical Optimisation 17

2.1.1 Gate Sizing as the Primary Timing Lever 17

2.1.2 The Fixed-Topology Assumption 18

2.2 Learning-Based Gate Sizing 18

2.3 The Structural Ceiling of Sizing-Based Repair 19

2.3.1 A Delay Model for a Single Logic Stage 19

2.3.2 A Worked Example: Depth Reduction Through Restructuring . . . 20

2.4 The Logic-Physical Disconnect and the Case for Post-Layout Restructuring 22

2.4.1 Why Structural Violations Survive Into the Layout 22

2.4.2 Why Use Real Physical Data Rather Than Predict It 23

2.5 Related Work: Physically-Aware Logic Restructuring 23

2.5.1 CPA-remap [3] 23

2.5.2 Simulated-Annealing Resynthesis in OpenROAD [4] 24

2.5.3 Search-Based Logic Synthesis 25

2.5.4 Positioning of Our Work 26

2.6 Our Goal: Physically-Aware Cone Resynthesis 27

2.6.1 The Cut - Resynthesise - Reinsert Idea 27

2.6.2 The Layout-Preservation Invariant 27

2.6.3 Coupling Structure and Physics in One Loop 28

3 MCTS-Based Workflow: Design and Implementation **29**

3.1 The Role of the Upstream `rmp` [4]Module in Our Framework 30

3.1.1 What We Reuse 30

3.1.2 What We Built on Top 30

3.2 Overview of Our Entire Framework 31

3.2.1 Pipeline Stages and the End-to-End Contract . . . 32
3.2.2 Endpoint Selection and Baseline Establishment . . . 32
3.2.3 Cone Extraction . . . 33
3.2.4 Recipe Synthesis on the Extracted Cone . . . 34
3.2.5 Per-Candidate Physical Evaluation . . . 36
3.2.6 Commit-Time Stitch-Back and Final PnR + resizer . . . 37
3.2.7 External Functional-Equivalence Verification . . . 38
3.3 MCTS Search Over ABC Recipes . . . 40
3.3.1 Node Structure . . . 40
3.3.2 UCB1 Child Selection . . . 41
3.3.3 One MCTS Iteration . . . 43
3.3.4 Evaluation Cache . . . 45
3.3.5 Stall Detection and Restart . . . 45

4 Experimental Results and Analysis 47

4.1 Experimental Setup . . . 47
4.2 Validation: Logic Restructuring Beyond the Resizer . . . 48
4.3 End-to-End Timing Improvement . . . 51
4.4 Physical Evaluation: DPL + RC vs Original RMP . . . 55
4.5 Effect of the Percentage Filter . . . 58
4.6 Full PPA Analysis After Global Routing . . . 59
4.7 Complete Analysis on GCD Design . . . 61
4.8 Summary . . . 63

5 Conclusion and Future Works 65

5.1 Future Works in Consideration . . . 66

References 67

List of Figures

1.1 EDA Flow Overview [5]. 3
1.2 OpenROAD Flow [1]. 6

2.1 Overview of the RL-Sizer gate sizing framework utilizing the DDPG reinforcement learning algorithm [6] (Image taken from [6]). 19
2.2 Two functionally identical implementations of the same four-input function, sharing the `OAI211` complex gate (highlighted). (a) The critical signal (red) passes through extra inverting stages: five-stage critical path, delay floor $\approx 5\,d_{\text{int}}$. (b) The restructured form removes the redundant inversions and re-expresses the logic: three-stage critical path, floor $\approx 3\,d_{\text{int}}$. Gate sizing cannot transform (a) into (b); only logic resynthesis can. 21
2.3 Overall workflow of CPA-remap (Image taken from [3]) 24
2.4 Physically-aware cone resynthesis. (a) Placed design with the critical fan-in cone highlighted. (b) The cone is cut out, freeing its region; all other instances retain their positions. (c) The resynthesised cone is reinserted and legalised incrementally, leaving the surrounding layout unchanged. Unlike a re-PnR flow such as CPA-remap [3], the layout outside the cone is invariant by construction. 28

3.1 End-to-end flow of the MCTS-driven post-CTS resynthesis framework. A post-CTS design is read into OpenDB along with its parasitics; violating endpoints are collected from the STA graph; a MCTS loop searches the space of ABC GIA-operator scripts using UCB1 tree selection, evaluating each candidate by extracting a logic cone, re-mapping it under a real per-cone delay budget, incrementally placing the inserted cells with OpenDP, refreshing parasitics only on dirty nets, and scoring the result with a TNS cost; the winning script is then committed and a single PnR + resizer pass is performed. 31

3.2 Cone Extraction process. Once the cone got extracted the space of the instance that are in the cone leaves empty spaces in the layout. 34

3.3 Logic resynthesis flow within the restructure framework. The extracted logic cone is converted into an And-Inverter Graph (AIG), optimized using synthesis recipes to achieve the target delay, and technology-mapped before layout reintegration. 36

3.4 Per-candidate physical evaluation in the MCTS search 38

3.5 The best cone gets stitch backed into the layout and incrementally placed. After the final Single PnR + resizer step is made to obtain the final TNS score 39

3.6 Summarised visual workflow with an example design 40

3.7 One MCTS iteration over ABC recipes. **Selection** (blue) descends the tree by UCB1; **Expansion** (green) adds one untried operator as a new child; **Evaluation** (orange) scores the new recipe via the physical evaluator (DPL + Steiner RC + TNS); **Backpropagation** (red) updates visit counts and scores back to the root. Here n denotes each node's visit count — the higher n on `&syn4` shows UCB concentrating the search budget on the most promising subtree. 44

4.1 WNS improvement in percentage 53

4.2 TNS improvement in percentage 53
4.3 GCD slack value comparison over all the endpoints 61
4.4 GCD slack improvement over all the endpoints 62
4.5 Average Slack improvement over type of endpoints 62

List of Tables

1.1 Stages of the Physical Design Flow 6

1.2 Major Modules in the OpenROAD Flow 7

1.3 Common Timing Optimization Techniques 11

4.1 Summary of evaluated benchmark designs and their node counts. 48

4.2 Per-stage timing — Our Implementation. S1 = Baseline, S2 = After Repair, S3 = Restructure Only, S4 = Restructure+Repair (Final). Δ = S4−S1, positive = improvement. Seed = 3500 (JPEG/Eth-MAC: 500 iters; others 1500). 50

4.3 Timing improvement (%) and runtime — Our Implementation. WNS%/TNS% = improvement over the post-CTS baseline. Seed = 3500. 51

4.4 Timing improvement and runtime: Original RMP vs Our Implementation. ΔWNS/ΔTNS in ps over the post-CTS baseline; RT = wall-clock runtime. Seed = 3500. **Bold** = better ΔTNS. 56

4.5 Full PPA after `global_route + estimate_parasitics -global_routing + repair_timing` at best pct per design (Baseline vs Our Implementation). WNS/TNS in ps; area in μm^2; power in W. **Green** = improvement over baseline. 59

Chapter 1

Introduction

1.1 Electronic Design Automation Overview

Electronic Design Automation (EDA) refers to a comprehensive category of software tools, algorithms, and methodologies used to design, simulate, verify, and manufacture electronic systems, such as Integrated Circuits (ICs) and Printed Circuit Boards (PCBs) [7] . EDA automates the complex, multi-step design flow required to translate high-level behavioral or algorithmic specifications into a fully verified, manufacturable physical layout (GDSII).

In modern VLSI design, EDA is essential for managing billions of transistors, ensuring the design adheres to stringent manufacturing rules, and meeting targeted constraints for Performance, Power, and Area (PPA).

In modern Very Large Scale Integration (VLSI) design, the foundational methodology is the RTL-to-GDSII flow, where Electronic Design Automation (EDA) tools synthesize high-level Hardware Description Language (HDL) behavioral models into a manufacturable physical layout. However, this automated transformation is rarely a linear, single-pass process and does not inherently guarantee timing closure. Due to stringent constraints on Performance, Power, and Area (PPA), the physical design cycle is highly iterative. Designers must fre-

quently backtrack to earlier synthesis or placement stages to perform further optimizations and resolve timing violations, ensuring final design convergence.

The design flow can be divided into three major parts, namely, front-end design, back-end design, and sign-off verification.

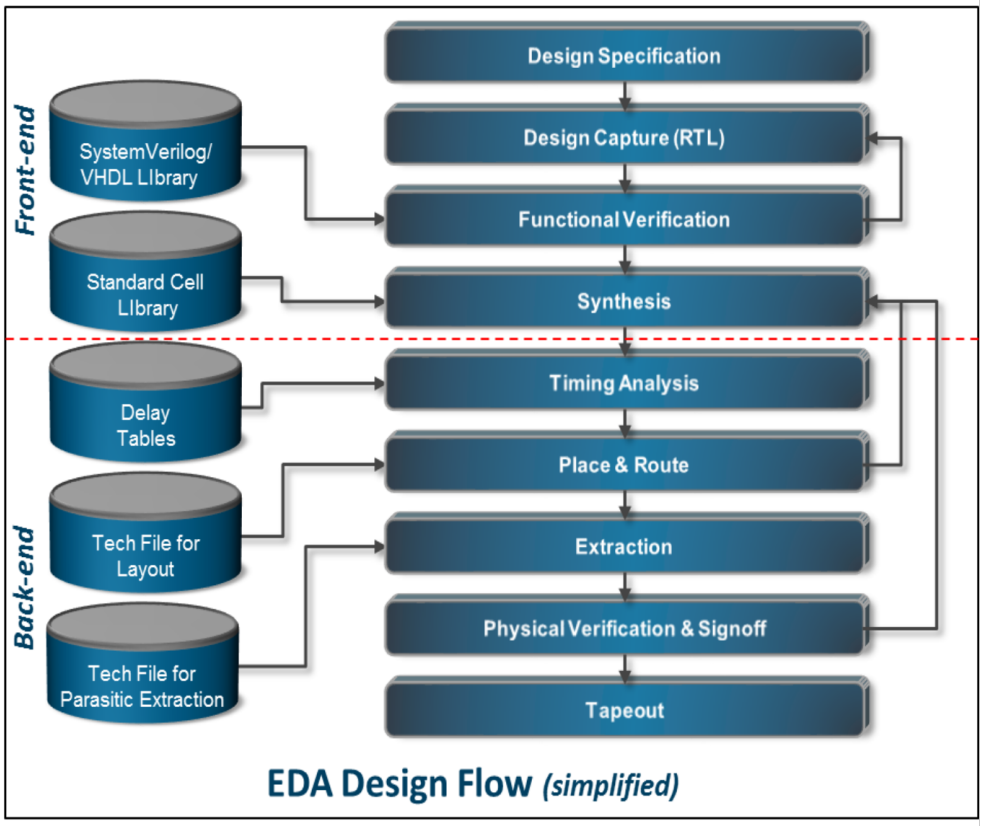


Fig. 1.1: EDA Flow Overview [5].

1.1.1 Front-End Design

The first step, often called front-end design or logical design, is all about figuring out how the chip is supposed to work. It starts with RTL design, where engineers lay out the circuit's logic and how data flows. After that, the RTL code goes through a round of functional verification, which means running a lot of simulations to make sure everything works as it should. Once it passes that check, the design moves on to logic synthesis. This is where the abstract RTL gets transformed into a gate-level netlist, made up of standard logic cells

like AND, OR, and flip-flops. These cells come from a standard cell library provided by the semiconductor foundry. At this point, they also add Design for Test (DFT) structures to ensure that the final chip can be tested after it's made.

1.1.2 Back-End Design

The second stage, often referred to as back-end design or physical implementation, focuses on turning the gate-level netlist into a real silicon layout. It starts with floorplanning, where the major functional blocks, like processor cores and memory units, get their spots on the chip. Next up is placement, which is all about carefully arranging millions of standard cells on the die while keeping timing and area in check. After that, we move on to clock tree synthesis (CTS), which makes sure the clock signal gets evenly distributed throughout the chip, so all the sequential elements get it with minimal delay. Lastly, we have routing, where we set up metal connections to link all the placed cells, finishing off the physical design.

1.1.3 Sign-Off and Verification

In the last phase of the process, which involves sign-off and verification, we make sure the design is not only correct but also ready for manufacturing. The key part of this phase is static timing analysis (STA), where we check that all timing requirements are satisfied by examining the setup and hold conditions for every circuit path. At the same time, we carry out physical verification to confirm that the layout adheres to the manufacturing design rules and that the physical connections match the logical design. This includes checking design rules (DRC) and comparing layouts to schematics (LVS). Once we get the green light on these verifications, we create the final GDSII file—a step known as tape-out. This file acts as the ultimate blueprint that gets sent off to the semiconductor foundry for production.

1.2 Introduction to OpenROAD

Within the EDA flow, physical synthesis bridges the gap between logical and physical domains. Rather than simply translating RTL into logic gates, physical synthesis takes a gate-level netlist and optimizes it concurrently with spatial placement, ensuring that all logic transformations are fully aware of physical layout parameters and wire parasitics [7] .

OpenROAD is a comprehensive, open-source toolchain designed for end-to-end physical design. Its primary goal is to make Integrated Circuit (IC) design more accessible, affordable, and open for both research and commercial innovation [1].

A key capability of OpenROAD is its autonomous RTL-to-GDSII flow, often called a "No-Human-in-loop" (NHIL) flow. This feature allows the tool to automatically generate a complete physical layout (in GDSII format) from an RTL description in less than 24 hours. The process works by taking a synthesized netlist and a set of constraints as input and producing a fully routed GDSII file as output.

To support different needs, OpenROAD provides a flexible API with bindings for both Tcl and Python, allowing users to customize the design flow. OpenROAD is actively used in university courses, global training programs, and major chip manufacturing initiatives like the Google-SkyWater and GlobalFoundries shuttle programs.

To make the tool easier to use, the OpenROAD-flow-scripts (ORFS) are provided. ORFS supports two modes of operation: fully autonomous execution for a hands-off approach, and manual control, using Tcl commands or Python APIs.

1.3 Workflow of OpenROAD [1]

The OpenROAD tool implements the complete physical design process through a series of well-defined stages. To execute this flow, it uses a set of integrated modules, all accessible through a unified database and user interface.

1.3.1 The Main Stages of Physical Design

The primary sequence for a physical design implementation in OpenROAD is as follows:

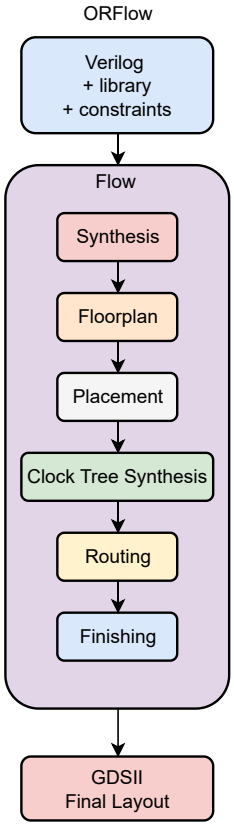


Fig. 1.2: OpenROAD Flow [1].

Stage	Description
Floorplanning	Defining the chip's area, and placing I/O pins and large blocks (macros).
Global Placement	Finding an initial, rough position for all standard cells.
Detailed Placement	Adjusting cell positions to their legal positions and to optimize timing and congestion.
Clock Tree Synthesis (CTS)	Building and buffering the clock network to deliver the clock signal to all sequential cells with minimal skew.
Global Routing	Planning the general paths for all signal wires across the chip.
Detailed Routing	Connecting the actual wire segments and vias to complete all logical connections.
Chip Finishing	Final steps, such as adding filler cells and performing final design rule checks (DRC).

Table 1.1: Stages of the Physical Design Flow

The process starts with floorplanning, which lays out the chip area, positions big macros, and determines where the pins go creating the basic framework for the layout. Then we move on to global placement, which organizes all the standard cells so we can get initial estimates on timing and congestion, and after that, we get into detailed placement to fine-tune cell positions and make sure everything's legal while optimizing wire usage. We also do clock tree synthesis (CTS) to make sure the clock is distributed with minimal skew and balanced loads throughout the design.

After we've placed everything, we shift into global routing. This phase plans out the

wire paths, keeping congestion and performance in mind, and then we dive into detailed routing to connect all the nets using the right metal layers and vias to complete the circuit connections. Finally, we wrap things up with chip finishing, where we add filler cells and run thorough design rule checks (DRC) to ensure everything can be manufactured properly. OpenROAD's combined scripting environment automates each of these stages, giving users the chance to optimize for power, performance, and area (PPA) just by tweaking flow parameters or getting involved in crucial steps if necessary. This makes OpenROAD a great option for both academic research and prototyping, and it's becoming more relevant for large-scale chip design in the industry.

1.3.2 Integrated Core Modules

To accomplish above stages, OpenROAD integrates several specialized modules, including:

Module	**Description**
OpenDB	Unified database for reading and writing design data.
Floorplanning	Tools for creating the initial floorplan.
TritonPlace / RePlace	Modules used for global placement.
OpenDP	Module handling detailed placement.
TritonCTS	Performs clock tree synthesis.
FastRoute	Engine used for global routing.
TritonRoute	Tool used for detailed routing.
Resizer	Gate sizing and buffer insertion for timing closure.
OpenSTA	Static timing analysis engine.
OpenRCX	Parasitic extraction module.

Table 1.2: Major Modules in the OpenROAD Flow

At the centre of the ecosystem sits OpenDB, which is this unified database engine that keeps track of all the physical and logical design data. It's really the backbone that allows all the different stages to work together smoothly. For floorplanning, there are specific modules dedicated to defining chip areas, arranging inputs/outputs, and managing macro placements. When it comes to cell placement, the process uses TritonPlace and RePlace for broader strategies, plus OpenDP for the finer details and making sure everything complies

with design rules and timing needs.

Then, there are specialized engines handling the later stages. For instance, TritonCTS is responsible for creating clock trees that balance load while minimizing skew. FastRoute and TritonRoute take care of global and detailed routing, addressing congestion and sticking to the intricate design rule constraints. To support timing and signoff, there's OpenSTA for static timing analysis, Resizer for gate sizing and buffer insertion, and OpenRCX for extracting parasitics. Each of these modules can work on their own or as part of the bigger picture, which makes for automated error detection and offers users flexibility in configuration. Plus, it directly optimizes performance, power, and area (PPA). This modular approach makes OpenROAD super adaptable, accommodating various process technologies and allowing users to customize and enhance it for either research or industrial production.

1.3.3 Support Tools

The OpenROAD flow is more than just a straightforward series of steps; it's actually a pretty complex, iterative process. This iteration relies on a set of analysis and support tools that help steer optimization and ensure that the design is accurate. At the heart of this are two key analysis tools: OpenSTA for timing and OpenRCX for parasitic extraction.

OpenSTA [8] plays a crucial role in timing optimization within this flow. Rather than running simulations, it statically evaluates every possible path in the design to check for setup and hold violations, figuring out the Total Negative Slack (TNS) and Worst Negative Slack (WNS). It's an open-source timing analyzer that the OpenROAD project uses, and it's well-built—originating from the Parallax timing engine, which has been around commercially for almost twenty years, used by numerous EDA companies. OpenSTA is freely available, accommodates advanced foundry nodes, and produces standard timing reports. It serves as the authoritative timing engine, continuously evaluating the design against specified constraints and providing critical feedback to guide other physical design stages,

such as placement, Clock Tree Synthesis (CTS), and optimization.

OpenRCX [9] . Before OpenSTA can do its thing, it needs to understand the delays caused by wires, which is where parasitic extraction comes into play. So, what exactly are parasitics? In a perfect, theoretical design, wires would be flawless, with zero delay. But in real silicon chips, the metal wires that connect the standard cells have physical characteristics that can slow down signals. These characteristics are known as parasitics:

Resistance (R): Every little metal wire has some resistance that opposes current flow and drains power.

Capacitance (C): Each wire functions like a small capacitor, storing charge. It has capacitance with the wires above and below it, as well as coupling capacitance with adjacent wires, and ground capacitance with the silicon substrate.

Why do we care about parasitics? Well, signals can only travel down a wire as quickly as they can charge up these tiny parasitic capacitors through the wire's resistance. This RC delay contributes to the overall interconnect delay. At advanced technology nodes (like 7nm or 5nm), this delay becomes a significant factor in the total delay of a path, sometimes even outweighing the switching delays of the gates. If you don't have accurate parasitic data, any timing analysis you do is basically flawed and not very useful.

Here's where OpenRCX steps in. It's the parasitic extraction tool used in OpenROAD, designed to extract those R and C values from the physical layout. It takes the foundry's Process Design Kit (PDK) to create linear regression models for wire resistance, ground capacitance, and coupling capacitance. After routing, OpenSTA reaches out to OpenRCX, providing the current layout in a DEF file. OpenRCX assesses the geometries of the wires and their surroundings and then produces a standard SPEF (Standard Parasitic Exchange Format) file. This file contains all the extracted parasitic R and C values for each net, which OpenSTA then uses to carry out a precise, "signoff-quality" timing analysis.

1.3.4 Visualization and Control

OpenROAD provides a powerful Graphical User Interface (GUI) that offers essential visualization and debugging features. This allows a user to graphically analyze:

- Floorplan and macro placement views
- Placement congestion maps
- Clock tree structures
- Global and detailed routing paths

Alongside the GUI, the tool provides a customizable Tcl interface. This enables script-based control and allows for more detailed, automated analysis of the design.

1.4 Timing Optimization

Primary focus of this project is timing optimization, also known as timing repair. This is the critical process of fixing timing violations failures to meet setup and hold constraints that prevent the design from operating correctly. In digital design, setup time dictates that data must arrive before the clock edge, while hold time dictates that data must remain stable after the clock edge. Timing repair becomes essential after placement because initial synthesis relies on rough wire delay estimations. Once the cells are placed and routed, the actual wire Resistance and Capacitance (RC) values are known, often causing new violations that were not visible earlier. The goal of timing repair is to then physically and electrically adjust the design, aiming to bring the Worst Negative Slack (WNS) and Total Negative Slack (TNS) to zero or near-zero. This must be accomplished without violating other electrical limits (like max slew, capacitance, or fanout) to ensure the final design operates reliably at its target clock frequency.

1.4.1 Techniques in use for Timing Optimization [2]

Timing repair in OpenROAD uses several optimizations to fix violations, affecting setup/hold timing, slew, capacitance, power, and area. The primary techniques include:

Optimization	Description
Gate Sizing (Upsize/Downsize)	Changing a cell's drive strength by changing the size of the gate to make it faster (fixing setup) or slower (fixing hold).
Buffer Insertion	Adding buffers to break long wires to improve signal delay and slew.
Gate Cloning	Duplicating cells that drive too many gates to reduce electrical load.
Cell Swapping	Replacing a cell with another with same functionality but better timing characteristics (e.g., faster but larger).
Re-buffering	Reorganizing existing buffers to find a better balance for delay and signal integrity.

Table 1.3: Common Timing Optimization Techniques

1.4.2 Gate Resizing

Gate resizing is one of the most fundamental and effective techniques for timing repair. The core concept is to swap a logic gate in the design with another gate that performs the exact same logical function (e.g., a 2-input AND gate) but has a different drive strength. Standard cell libraries (the .lib files) provide multiple versions of each gate, such as AND2_x1, AND2_x2, and AND2_x4, where a higher number typically means a larger, more powerful cell.

Upsizing: This means swapping a cell for one with a higher drive strength (e.g., NAND2_x1 $\rightarrow$ NAND2_x2). A stronger cell can drive signals faster, which reduces its propagation delay. This is the primary method used to fix setup violations.

Downsizing: This means swapping a cell for one with a lower drive strength (e.g., NAND2_x4 $\rightarrow$ NAND2_x2). A weaker cell is slower and smaller. This is used to fix hold violations (by adding delay) and to recover power and area on non-critical paths.

The Resizer (Timing Repair) Process in OpenROAD

In OpenROAD, timing repair and design rule optimization are managed by the `Resizer` module. Rather than relying on a single isolated command, the Resizer executes a suite of physical synthesis transforms—primarily driven by the `repair_design` (for electrical violations) and `repair_timing` (for slack violations) Tcl commands. The module acts as an optimization engine tightly integrated with OpenSTA (Static Timing Analysis) and OpenDB (the physical database) in an iterative feedback loop:

1. **Parasitic Estimation & Initial STA:** Before any optimization, OpenROAD estimates wire parasitics using either a Wire Load Model (WLM) or global routing topologies. OpenSTA is called to perform a full static timing analysis, identifying Design Rule Violations (DRVs) such as max capacitance and max transition/slew, alongside timing violations (Worst Negative Slack, Total Negative Slack).

2. **Design Rule Violation (DRV) Repair:** Prior to optimizing timing paths, the Resizer fixes electrical violations. High-fanout nets and nets exceeding maximum capacitance or transition limits are repaired by building optimized buffer trees or inserting localized buffers.

3. **Critical Path Tracing:** For timing repair, the Resizer queries OpenSTA to identify violating endpoints. It traces backward through the critical logic cones driving these slack violations to isolate candidate gates and nets for physical synthesis transforms.

4. **Multi-Strategy Transform Selection:** The Resizer does not limit itself to gate sizing alone; it evaluates multiple optimization strategies along the critical path:

    - **Gate Sizing:** Upsizing cells to increase drive strength and reduce delay for setup fixes, or downsizing non-critical cells for area and power recovery.

    - **Buffer Insertion:** Breaking long, highly-resistive wires or high-capacitance nets by inserting buffers to minimize propagation delay.

- **Pin Swapping:** Swapping logically equivalent pins on a gate to assign the critical timing signal to the input pin with the fastest internal propagation delay.
- **Gate Cloning:** Replicating a driving gate to split a heavy load across multiple parallel paths, reducing the capacitive load on the original gate.

5. **Physically-Aware Cost-Benefit Analysis:** For every proposed transform, the Resizer evaluates the timing delta against physical constraints. It verifies whether an target cell change or buffer insertion has a legally available placement site and calculates the localized parasitic impact to ensure the change does not introduce new hold, setup, or DRV violations on neighboring nets.
6. **Incremental Layout Update & Iteration:** Once a valid, beneficial transform is identified, the netlist and physical layout are updated incrementally in OpenDB. The internal incremental placer ensures all modified or newly inserted cells are legalized.

This closed-loop process (STA $\rightarrow$ Transform Selection $\rightarrow$ Physical Verification $\rightarrow$ Legalization $\rightarrow$ Incremental STA) repeats iteratively. The Resizer progressively mitigates WNS and TNS until timing converges, the structural transformation constraints are met, or a pre-defined iteration limit is reached.

1.4.3 Limitations of Current Timing Repair Methodologies

The `Resizer` module in OpenROAD, while useful, has some fundamental limitations when dealing with severe timing violations.

Topological Rigidity of the Netlist

The repair process cannot change the netlist topology produced during synthesis. It is restricted to cell sizing and buffer insertion along existing paths, meaning any timing problem rooted in the logic structure itself cannot be fixed.

Inability to Optimize Critical Fan-In Cones

Since `Resizer` processes gates one at a time, it has no view of the complete fan-in cone driving a violating endpoint. This makes it impossible to refactor or restructure the underlying Boolean logic into a more timing-friendly form.

Diminishing Returns of Sizing and Buffering

Repeatedly upsizing cells increases input capacitance on earlier stages, which can create new violations upstream. Beyond a certain point, sizing and buffering alone cannot close the slack gap.

Lack of Physically-Aware Structural Resynthesis

There is no feedback loop between the physical layout and logic restructuring. The tool cannot identify a problematic timing cone, rewrite its logic with awareness of wire delays, and stitch the result back into the placed design.

1.5 Contribution

This thesis proposes a logic restructuring framework that works directly on a post-placement netlist to fix timing violations by combining logic-level changes with physical timing repair.

- Critical fan-in cones driving the selected violating endpoints are extracted and passed to ABC along with the target delay, which rewrites them into faster structural equivalents.
- A Monte Carlo Tree search is used to find the best sequence of ABC restructuring commands for each cone, since no single command works well across all cases.
- The search is guided by a score derived from the TNS obtained after the rewritten cone is stitched back into the design and incremental placement and parasitic estimation

are performed.

- A candidate is accepted only if incremental STA shows a strict improvement over the baseline, ensuring the framework never makes timing worse. Once the best candidate is committed, a final placement and routing pass is performed to legalise the layout with the newly inserted cone, followed by a `repair_setup` run to fix any residual violations.

1.6 Report Organisation

This thesis consists of four chapters:

Chapter 2 reviews prior work, covering how gate sizing is used and the limitations of gate resizing and how the logical restructuring is needed and how the CPA-Remap has drawn the motivation for the same.

Chapter 3 details the methodology and implementation steps of the proposed framework. Specifically, it discusses the adaptation of components from the rmp [4] module within the OpenROAD toolchain, namely the logic extraction mechanisms and the ABC-based resynthesis engine. Building upon these foundational utilities, this work develops a comprehensive, fully integrated framework that ensures the entire logic restructuring process is physically-aware and guided by spatial constraints.

Chapter 4 presents results, comparing our results to the Openroad's timing repair over a set of designs with various sizes of cone extraction, also showing how the improvement is visible on the design.

Chapter 5 concludes the thesis by summarizing the work presented and discussing potential future directions across the different phases of the proposed framework.

Chapter 2

Reveiw of Prior Works

In Chapter 1, we discussed the role of post-placement timing repair and `Resizer`, the engine that implements it in OpenROAD. This chapter develops the dissertation's central argument in five movements. First, we bound, in a charitable way, what electrical optimisation can achieve by looking at gate sizing at its learning-based frontier (§2.1–§2.2). We then show, by construction, that this whole family of transforms has a hard structural ceiling (§2.3). We attribute structural violations to the disconnect between logic synthesis and physical implementation, and argue that the remedy must be applied *post-layout* (§2.4). We review the recent physically-aware restructuring work that shares our problem statement and position our contribution with respect to it (§2.5. We conclude by formalizing the physically-aware cone-resynthesis objective that the thesis achieves (§2.6).

2.1 The Scope and Limits of Electrical Optimisation

2.1.1 Gate Sizing as the Primary Timing Lever

Gate sizing is widely used in electronic design automation; its main idea is to tune circuit timing strictly through electrical adjustments rather than structural logic changes. As the primary lever for delay control in physical synthesis, sizing involves replacing a specific cell

with a logically equivalent variant of a different drive strength. Upsizing resolves setup violations by lowering output resistance to drive fixed capacitive loads faster, whereas downsizing mitigates hold violations and recovers area and leakage power. The broader suite of physical synthesis tools—including buffer insertion, gate cloning, re-buffering, and pin-swapping—operates on similar principles. They redistribute load and optimize electrical characteristics along a critical path, yet they are fundamentally constrained: they cannot change the Boolean function being computed, nor can they reduce the number of logic stages.

2.1.2 The Fixed-Topology Assumption

The common property of this family is that it works on a *fixed netlist topology*. The gates, their interconnection and decisively the number of combinational stages between launch and capture registers are all inherited unchanged from the synthesis. Sizing selects a point in the discrete space of drive-strength assignments defined *over* that fixed graph, never editing the graph. This is what makes electrical optimisation fast, local and incrementally safe. It is also what limits it. When the violation is born of the structure and not the sizing, the optimiser searches the wrong space.

2.2 Learning-Based Gate Sizing

To rigorously evaluate gate sizing beyond simplified, theoretical models and approach the current research frontier, it is instructive to examine RL-Sizer [5]. This modern framework reformulates the gate sizing challenge as a deep reinforcement learning problem, specifically driven by the Deep Deterministic Policy Gradient (DDPG) algorithm. Each candidate gate gets encoded as a *state* by running its local multi-hop neighbourhood through a graph neural network and appending technology features to it. An *action* is just a drive-strength change Δd, snapped to the nearest legal cell in the library. The *reward* is whatever improvement in local Total Negative Slack you get after applying that change and re-timing the path.

A DDPG agent then learns a policy that keeps accumulating slack recovery over time. What's genuinely impressive about RL-Sizer is that it's placement-aware, neighborhood-aware, and it finds trade-offs that hand-tuned heuristics consistently miss. That's not a small thing. But here's the catch. Its action space is, by construction, confined to drive-strength assignments on a fixed topology. No matter how well the agent trains, it can only choose *sizes* never a different *structure*. You can't learn your way out of a paradigm you're locked inside. And this isn't really a critique of RL-Sizer specifically; it's just the ceiling of the sizing paradigm as a whole. How low that ceiling sits is something the next section puts some numbers to.

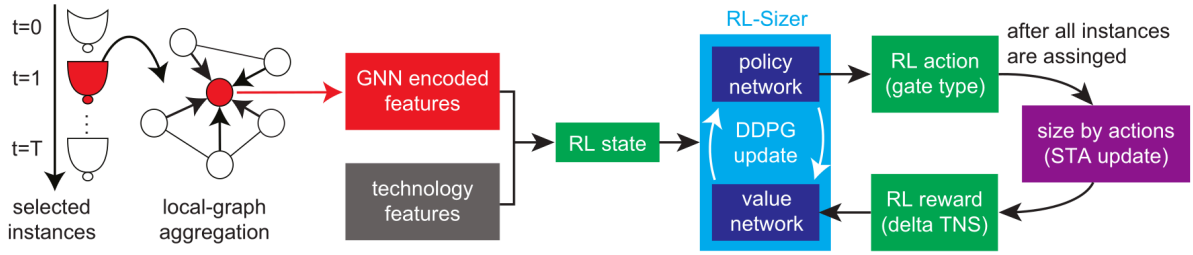


Fig. 2.1: Overview of the RL-Sizer gate sizing framework utilizing the DDPG reinforcement learning algorithm [6] (Image taken from [6]).

2.3 The Structural Ceiling of Sizing-Based Repair

2.3.1 A Delay Model for a Single Logic Stage

A first-order model of a combinational stage driving a load writes its delay as

$$d \;=\; d_{\text{int}} \;+\; R_{\text{drv}}\, C_{\text{load}},$$

where d_{int} is the intrinsic, parasitic-limited delay (a property of the cell, independent of load) and $R_{\text{drv}}\, C_{\text{load}}$ is the load-dependent component. Sizing acts only on the second term: upsizing lowers R_{drv} but raises the input capacitance the *previous* stage must drive, shifting load upstream rather than removing it. In the limit of arbitrary drive strength the load

term can be driven toward zero, but d_{int} cannot be reduced at all. Equivalently, logical-effort theory writes the minimum delay of an N-stage path as $D_{\min} = N\,(GH)^{1/N} + P$ with $P = \sum_i p_i$ the total parasitic delay; sizing tunes only the electrical efforts and cannot touch P, which grows with depth N. *Depth sets a delay floor sizing cannot breach.*

2.3.2 A Worked Example: Depth Reduction Through Restructuring

Figure 2.2 makes the argument concrete. Both circuits compute the *same* four-input function and share the same `OAI211` complex gate (highlighted); they differ only in how the critical signal (red) reaches the output. In Figure 2.2(a) it passes through a chain of extra inverting stages, so the critical path is five stages deep. In Figure 2.2(b) those redundant inversions have been removed and the surrounding logic re-expressed, leaving a critical path of three stages that computes exactly the same Boolean output.

The consequence follows directly from the stage model of §2.3. Each circuit's critical-path delay is floored by the sum of its stages' intrinsic delays, so (a) cannot go below $\approx 5\,d_{\text{int}}$ while (b) is floored at $\approx 3\,d_{\text{int}}$. The $2\,d_{\text{int}}$ gap between them is purely a difference in *stage count*. Sizing cannot close it: upsizing the gates in (a) shrinks the load-dependent delay within each stage but leaves the two extra stages in place, and every upsize loads the previous stage more, so the gains saturate while the floor stands. The only way to reach (b) is to *change the structure* delete the redundant inversions and fold logic into adjacent gates. That is exactly what logic resynthesis does: ABC's balancing, rewriting, and technology remapping search the And-Inverter-Graph for a lower-depth realisation of the same function, turning (a) into (b) and removing the delay sizing could never touch.

The example is small, but what it is pointing at is general. Whenever a path's binding constraint comes from its logic depth or its factored form, sizing, buffering, and cloning are essentially working on the wrong variable: they tune delay *within* each stage, but cannot change the *number* of stages. That is the root of the diminishing returns you see when

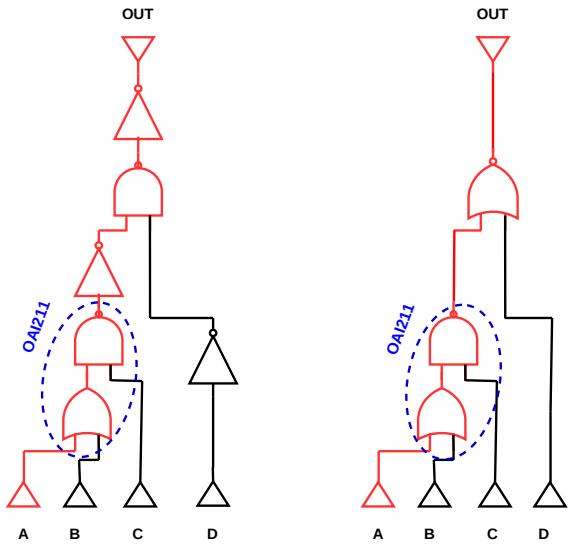


Fig 2.1(a) Fig 2.1(b)

Fig. 2.2: Two functionally identical implementations of the same four-input function, sharing the `OAI211` complex gate (highlighted). (a) The critical signal (red) passes through extra inverting stages: five-stage critical path, delay floor $\approx 5\,d_{\text{int}}$. (b) The restructured form removes the redundant inversions and re-expresses the logic: three-stage critical path, floor $\approx 3\,d_{\text{int}}$. Gate sizing cannot transform (a) into (b); only logic resynthesis can.

sizing-only repair runs into severe violations each successive upsize buys less delay reduction at a progressively higher cost in capacitance, area, and congestion, while the depth-set floor $N\,d_{\text{int}}$ just sits there, unmoved.

Resynthesis attacks the one variable sizing cannot touch: it changes N. By re-deriving the cone's logic into a shallower, better-factored structure, it pulls the delay floor down rather than endlessly chasing it from above. That is a fundamentally different kind of move. And this is what makes resynthesis *complementary* to electrical optimisation rather than a replacement for it sizing is still the right tool when dealing with drive-limited or load-limited paths, no argument there. But for the structurally-limited violations that tend to dominate hard timing closure, resynthesis is the only lever that actually reaches the root cause. Everything else is just symptom management.

2.4 The Logic-Physical Disconnect and the Case for Post-Layout Restructuring

A fundamental question arises: if initial logic synthesis already minimizes logic depth, why do structure-limited timing violations persist into the physical layout? The root cause is the well-documented disconnect between the abstract estimations available during early synthesis and the actual spatial realities—such as wire parasitics and placement congestion—present in the final design [3].

2.4.1 Why Structural Violations Survive Into the Layout

Logic synthesis commits to a structure while optimising against *estimated* circuit metrics. The wire-delay models used during technology mapping have, with advancing process nodes, become increasingly inadequate for accurate timing prediction [3]. So the structural decisions get frozen long before the information needed to make them correctly even exists. The gap only widens from there. At advanced nodes, post-layout interconnect delay increasingly dominates total path delay, sometimes overwhelmingly so. A path that synthesis believed was comfortably balanced can emerge from routing as *the* critical path purely because its interconnect contribution was mis-estimated [3]. That's not a small error. That's the entire premise of the optimisation being wrong. And it gets worse. Criticality isn't even purely logical. As placement spreads cells across the die under density and overlap constraints, tightly connected cells can get forced far apart. Those long interconnects then become critical-path segments in their own right [3]. Such a path is structurally critical in the physical sense no local sizing fixes it, because the problem isn't the strength of any one cell. It's the spatial arrangement of the group as a whole. In every case, the conclusion ends up being the same: the structure has to be revisited *after* layout, when the real timing is finally known. And the natural remedy is to extract the offending cone, restructure its logic, and re-place it.

2.4.2 Why Use Real Physical Data Rather Than Predict It

One school of physically-aware synthesis tries to close this loop differently by estimating post-layout metrics with machine-learning models and feeding them back into mapping. These methods are fast, But they inherit three weaknesses their predictions are probabilistic and carry inherent uncertainty, their quality depends heavily on training data and generalises poorly across designs and process nodes, and the training itself adds non-trivial overhead [3]. This thesis takes the opposite stance. Rather than predict the physics, it uses the *actual* post-layout physical state real placement, real parasitics as the ground truth for every restructuring decision. The price is that the technique operates late in the flow. The reward is that it's completely free of prediction error, which is exactly what makes the structural decisions trustworthy in a way that ML-predicted ones simply aren't.

2.5 Related Work: Physically-Aware Logic Restructuring

The argument above that structural resynthesis must be driven by real post-layout information is precisely the premise behind a recent line of work. Of these, CPA-remap [3] sits closest to the problem this thesis is addressing.

2.5.1 CPA-remap [3]

CPA-remap extracts critical paths and cell locations from a post-layout design, merges and partitions them into initial cones, and then *grows* each cone using a proximity principle: only cells whose Half-Perimeter Wirelength distance falls below a threshold get absorbed. This keeps every cone spatially coherent and ensures that wire-delay estimates inside it are actually trustworthy [3]. Each cone is then remapped by ABC under timing constraints and written back into the netlist. Two of its design choices are directly relevant to what this thesis does. The first is a somewhat non-obvious insight CPA-remap reports that *a logic cone cannot be treated as a naive sub-netlist.* An unconstrained delay-driven remap

will improve the cone's worst path, but it does so by sacrificing non-critical paths for area. And that sacrifice can introduce *new* violations back in the parent netlist [3]. The fix is to encode per-input delay offsets as constraints, so that ABC is forced to treat all paths in the cone as critical, not just the worst one. The second is a *conservative acceptance* policy: post-remap timing is compared against the pre-remap baseline, and any cone that degrades on any path gets rejected outright. No exceptions. This guarantees that every accepted cone strictly improves timing [3]. On OpenCores benchmarks and a 4-million-cell industrial design, CPA-remap reports TNS improvements ranging from 9.1% to 96.1%, which fairly convincingly establishes that post-layout cone restructuring is both effective and practical at scale.

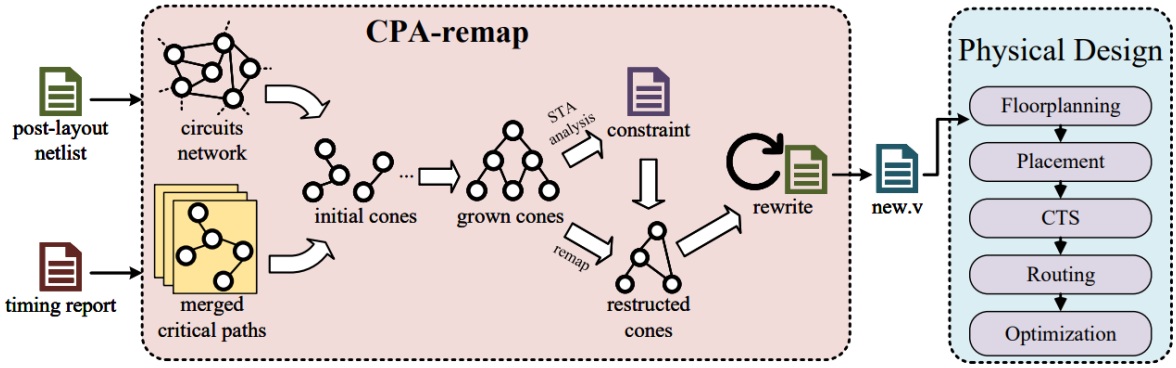


Fig. 2.3: Overall workflow of CPA-remap (Image taken from [3])

The overall flow described above is illustrated in Figure 2.3, which provides a visual summary of how each stage connects from the initial netlist and timing report inputs through to the final `new.v` output consumed by the physical design flow.

2.5.2 Simulated-Annealing Resynthesis in OpenROAD [4]

The most directly related prior work is the simulated-annealing resynthesis strategy developed by Antmicro and merged into the OpenROAD `rmp` module [10]. Like our framework, it operates inside OpenROAD and searches over sequences of ABC GIA operators using a Metropolis-style acceptance criterion. A neighbour solution is generated by adding, re-

moving, or swapping one operator in the current sequence, and the optimisation objective is the design's *worst negative slack* (WNS).

However, the approach has two fundamental limitations that our work addresses. First, the evaluation oracle is *physically blind*: after each RunGia call inserts the rewritten cone, WNS is read directly from cached STA without updating parasitics or legalising the new cells. Newly inserted cells carry zero wire delay in this state, creating an optimistic lower bound that causes the search to favour recipes which insert many unplaced cells rather than recipes that genuinely improve post-placement timing. Second, the optimisation objective is WNS only, so the search can improve the single worst path while worsening hundreds of other violating endpoints. Our framework replaces the stale-WNS oracle with a score derived from the full TNS measured *after* incremental cell legalisation and Steiner wire RC estimation, and uses MCTS rather than SA to exploit structural dependencies between ABC operators across iterations.

2.5.3 Search-Based Logic Synthesis

Several recent works frame logic synthesis as a search problem over sequences of ABC transformations. DRiLLS [11] trains a deep reinforcement-learning agent (A2C) to navigate the ABC operator space, achieving a 13% average QoR improvement on the EPFL arithmetic benchmark suite. AlphaSyn [12] applies Monte Carlo Tree Search with a domain-specific UCB variant (SynUCT), outperforming FlowTune by 8.74% on area-delay product.

A key distinction between these works and ours is the *design stage* and *evaluation oracle*. DRiLLS and AlphaSyn both operate pre-placement, where wire delays are estimated from simple wire-load models and the physical state of the design is not yet fixed. Their reward functions are computed on technology-mapped netlists with no placement or parasitic information. Our framework operates post-CTS on a fully placed design, and every candidate evaluation incorporates the real placement coordinates of the new cells and wire RC derived

from Steiner tree estimation on those coordinates. This is what makes the search signal physically meaningful: a recipe that reduces logic depth but produces cells far from their logical neighbours is correctly penalised, whereas the same recipe would appear attractive under a wire-load or zero-RC model.

2.5.4 Positioning of Our Work

Our framework shares the same core premise as CPA-remap: post-layout, cone-based, ABC-driven restructuring with conservative acceptance. The two approaches converge on several key safeguards independently, but differ decisively in how the physical state is handled after restructuring.

- Where CPA-remap constrains ABC to protect non-critical paths inside a cone, our framework uses a TNS-specific cost funtion that folds in the overall TNS value and a placement-legality penalty, so that we will be able to get the best cone which reduces the overall design slack.
- Where CPA-remap rejects any degrading cone outright, our framework anchors the search to a pre-repaired baseline and returns a guaranteed no-op when no candidate improves on it.
- CPA-remap is a netlist-level optimiser: after rewriting the cones, it emits a revised Verilog and re-runs the complete back-end flow from scratch, paying the cost of a full place-and-route cycle on every pass [3].
- Our framework never leaves the physical database. The restructured cone is stitched back into the placed design in the region the old cone vacated, legalized incrementally, and re-timed against parasitics refreshed only on the affected nets. The floorplan, clock tree, and routing on untouched paths are fully preserved.
- This makes our approach a late, surgical repair: it touches only the failing cones, leaves the rest of the design closure intact, and completes in a fraction of the time a

full back-end iteration would require.

- The Antmicro SA [10] is the closest prior work in the same code base, but evaluates candidates on unplaced cells with stale WNS — a physically blind oracle. Our framework replaces this with an MCTS-based search scored on TNS after incremental DPL and Steiner RC, giving the search a physically consistent signal that holds up after placement and routing.

2.6 Our Goal: Physically-Aware Cone Resynthesis

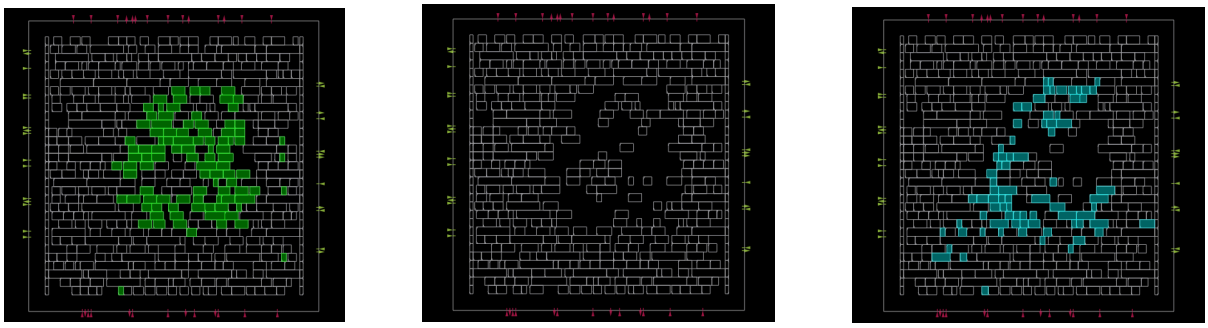
2.6.1 The Cut - Resynthesise - Reinsert Idea

Our objective is to perform structural resynthesis of a failing cone on a placed-and-routed design *while leaving the surrounding layout untouched*, as in Figure 2.4:

1. **Isolate** the combinational fan-in cone driving the violating endpoints (Figure 2.4(a)).
2. **Cut** that cone out, freeing the layout region its cells occupied, while every instance outside the cone keeps its exact placed coordinates (Figure 2.4(b)).
3. **Resynthesise** the cone's logic into a structurally superior, timing-budgeted variant and **reinsert** the new cells into the freed region, legalising *only* those cells incrementally (Figure 2.4(c)).

2.6.2 The Layout-Preservation Invariant

Rather than committing unplaced cells and relying on a global re-place to repair the layout, the freed cone region is treated as a small, local placement problem. New cells are seeded near the connectivity centroid of the cone they replace and legalised with an incremental detailed-placement pass that touches only those cells. Everything outside the cone is invariant by construction. And that invariant is what makes the technique actually usable inside a closed design a designer can apply it late, after routing, without replacing and



(a) Isolated (b) Cut out (c) Stitch back

Fig. 2.4: Physically-aware cone resynthesis. (a) Placed design with the critical fan-in cone highlighted. (b) The cone is cut out, freeing its region; all other instances retain their positions. (c) The resynthesised cone is reinserted and legalised incrementally, leaving the surrounding layout unchanged. Unlike a re-PnR flow such as CPA-remap [3], the layout outside the cone is invariant by construction.

re-routing the whole chip.

2.6.3 Coupling Structure and Physics in One Loop

The deeper consequence of the invariant is that it lets the structural search and the physical legalisation be *coupled in one loop*. Because each candidate structure can be dropped into the layout, legalised locally, and re-timed against refreshed parasitics, the optimiser evaluates a candidate not on some idealised netlist but on the real, legalised, parasitic-loaded design it would actually produce the property that upstream modules lack and the framework of the next chapter supplies.

Chapter 3

MCTS-Based Workflow: Design and Implementation

The `rmp`[4] module in OpenROAD is a logic-level resynthesis pass that was originally designed to run before the physical design stage. It provides some useful building blocks, such as the cone extraction logic and the ability to stitch a rewritten cone back into the netlist, which we reuse as a base for our framework. However, these primitives alone are not sufficient for our use case, since our framework operates on a fully placed-and-routed design and needs to stay physically consistent throughout. A significant amount of new logic had to be built on top of these primitives to make the flow work correctly.

The rest of this chapter walks through the full pipeline we implemented: how the MCTS search is set up, how each candidate is evaluated against real timing, how the best cone is committed back into the placed design, and how the result is verified.

3.1 The Role of the Upstream `rmp` [4]Module in Our Framework

3.1.1 What We Reuse

- **Cone extraction** via the backwards-BFS walker, which takes a set of violating endpoints and produces a logic cone with the correct boundary inputs and outputs.
- **The ABC GIA operator vocabulary** and the AIG-to-technology-mapped network conversion path.

3.1.2 What We Built on Top

- The original cone extraction code collects all failing endpoints and performs a BFS over the full set. Our framework restricts this to the worst K% of endpoints, which bounds the cone size and keeps per-iteration cost manageable. We also found that the original code passed a unit delay as the resynthesis target rather than the actual path requirement; this was replaced with the real target delay derived from the timing report. Several other small adjustments of the same nature were made to the borrowed code from the `rmp` [4]module to make it compatible with our flow.
- The SA loop was replaced with a Monte Carlo Tree Search (MCTS), which builds a tree over op-sequence prefixes and uses UCB1 selection to concentrate the iteration budget on the most promising recipe subtrees, replacing the original WNS-only random walk.
- Each candidate evaluation now includes an incremental parasitic refresh and an incremental legalisation step, so timing is read on a physically consistent state after every cone mutation.
- The final commit path seeds new cells with placement hints, runs incremental legalisation, refreshes parasitics, and performs a global placement pass to recover any whitespace introduced by the new cone. A subsequent `resizer` pass cleans up any

residual violations; in practice this additional step has consistently produced better final timing than committing without it.

- A functional equivalence check is run at the end to confirm that the rewritten cone computes the same logic as the original.

3.2 Overview of Our Entire Framework

The framework treats the placed-and-routed design as the ground truth, and each candidate resynthesis as a proposal that must demonstrably improve a TNS cost without breaking either placement legality or functional equivalence. Because every netlist change costs both detailed-placement effort and parasitic re-estimation, the framework is structured so that the expensive operations technology mapping, detailed placement, wire-RC re-estimation are scoped as tightly as possible per candidate, and so that any failure can be rolled back atomically through OpenDB's ECO journal.

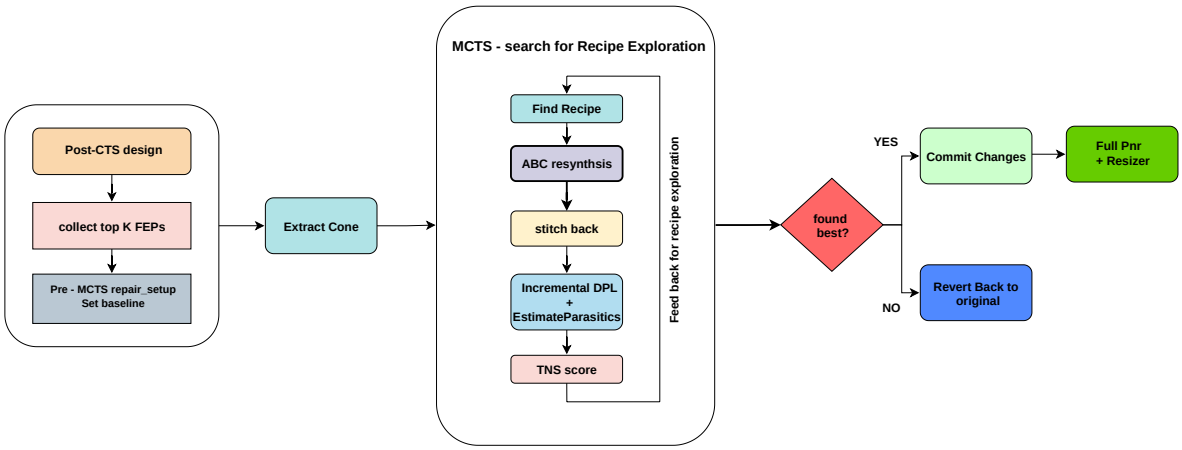


Fig. 3.1: End-to-end flow of the MCTS-driven post-CTS resynthesis framework. A post-CTS design is read into OpenDB along with its parasitics; violating endpoints are collected from the STA graph; a MCTS loop searches the space of ABC GIA-operator scripts using UCB1 tree selection, evaluating each candidate by extracting a logic cone, re-mapping it under a real per-cone delay budget, incrementally placing the inserted cells with OpenDP, refreshing parasitics only on dirty nets, and scoring the result with a TNS cost; the winning script is then committed and a single PnR + resizer pass is performed.

3.2.1 Pipeline Stages and the End-to-End Contract

The runtime of `resynth_mcts` proceeds through six stages:

1. **Endpoint selection** from the in-flow `dbSta` timing graph, with optional truncation to the worst $N\%$ of violators.
2. **Logic cone extraction** around the retained endpoints performed once globally.
3. **MCTS tree initialisation** — the root node represents the empty op-sequence (no restructuring). Each tree node stores the prefix of ABC operators applied so far, its visit count, and the best TNS score seen through that prefix. UCB1 selection balances exploitation of high-scoring prefixes against exploration of untried operators.
4. **ABC resynthesis** of the cone under an MCTS-selected GIA operator sequence, technology-mapped against a real per-cone delay budget.
5. **Per-candidate physical evaluation**: ECO-journal-wrapped insertion, slack-weighted centroid placement, incremental OpenDP legalization, incremental parasitic refresh on dirty nets, and composite-cost scoring; the journal is rolled back at the end of each evaluation so the loop's state is preserved.
6. **Commit of the winning script** under a final ECO journal that this time is endorsed rather than rolled back, followed by Placement and Routing and a closing `resizer` pass.

3.2.2 Endpoint Selection and Baseline Establishment

Endpoint selection. The framework walks the `sta::dbSta` endpoint set and retains a vertex only if (i) it is a flop or latch data-input pin or a top-level output port, (ii) it is not marked `dont_touch` on the pin, its net, or its instance, and (iii) its slack at the user's analysis corner is strictly less than the user-supplied threshold (default 0 s). The retained set is then optionally truncated to the worst $N\%$ of violators via the `percentage_`

parameter, which bounds the eventual cone size on designs with thousands of marginal endpoints.

SPEF-baseline capture. Before any wire-RC re-estimation runs, the framework records the *routed-wire* WNS as `spef_wns_baseline_`. This becomes the reference for the MCTS loop's hard regression guard. If the user has loaded SPEF, this is the real routed-wire WNS; if only wire-load is available, it is the pre-estimation WNS. Either way, it is the most conservative WNS the framework has access to, and the guard rejects any candidate that drops more than $10 \times |\text{ref}|$ below it.

3.2.3 Cone Extraction

Before the MCTS search begins we extract a logic cone from the current state of the design select a few set of endpoints which are failing and have high negative slack values rather than taking all the endpoints. Cone construction itself uses the upstream `cut::LogicExtractorFactory` partially changed according to our need: a backwards BFS from each retained endpoint collects every combinational cell reachable without crossing a sequential boundary, a top-level input port, or a `dont_touch` cell. The union of those cells, together with the nets crossing the resulting frontier, forms a single multi-rooted `LogicCut` whose primary outputs are the violating endpoints and whose primary inputs are the nets entering the cone from outside. This cut is the input to the next stage.

In our post-CTS setting the cone must be rebuilt inside every per-candidate evaluation, because the previous evaluation's ODB state is rolled back at the end of each iteration (see §3.2.5) and the cone must reflect the design state the next candidate is actually applied to. Cone extraction is cheap pure graph traversal, no STA queries so the per-iteration cost is dominated downstream by ABC mapping and OpenDP legalisation rather than by cone construction itself. The visualization example has been provide in the image below.

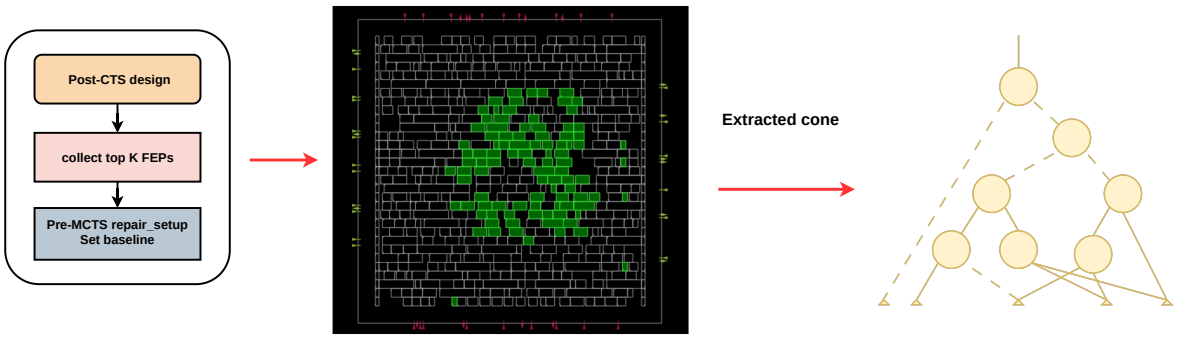


Fig. 3.2: Cone Extraction process. Once the cone got extracted the space of the instance that are in the cone leaves empty spaces in the layout.

3.2.4 Recipe Synthesis on the Extracted Cone

The cone is converted into ABC's compact AIG representation, the MCTS-chosen recipe is applied as an ordered sequence of GIA operators, and the result is technology-mapped against a real per-cone delay budget. Algorithm 1 summarises the steps.

Per-Cone Delay Budget

The single most important input to the mapper is the *delay budget*: the amount of time the cone's combinational logic is allowed to consume without violating the timing of the paths that pass through it.

Intuition. A logic cone sits between a set of inputs and a set of outputs. Its inputs do not all arrive at time zero — each is fed by upstream logic and arrives at some later moment. Likewise, each output has a *deadline*: the latest time its signal may settle so that the downstream flip-flop captures it correctly. The budget is simply the gap between the *latest* input to arrive and the *earliest* output deadline. It is the time window the cone has to do its work.

Formal definition. Let O_{cone} be the set of cone output pins (the endpoints the cone drives) and D_{in} the set of driver pins feeding the cone's inputs. From static timing analysis (STA) on the placed design we read, for every pin v:

- $T^{\mathrm{arr}}(v)$ — the *arrival time*: when the signal actually settles at v, propagated *forward* from the clock through all upstream logic and wire delay;
- $T^{\mathrm{req}}(v)$ — the *required time*: the latest v may settle without causing a setup violation, propagated *backward* from the capturing flip-flops.

(The familiar slack of a pin is $T^{\mathrm{req}}(v) - T^{\mathrm{arr}}(v)$; a negative slack is a violation.) The cone delay budget is then

$$\Delta_{\mathrm{cone}} \;=\; \underbrace{\min_{v \in O_{\mathrm{cone}}} T^{\mathrm{req}}(v)}_{\text{tightest output deadline}} \;-\; \underbrace{\max_{d \in D_{\mathrm{in}}} T^{\mathrm{arr}}(d)}_{\text{latest input arrival}} .$$

We take the *minimum* required time over outputs (the most urgent deadline the cone must meet) and the *maximum* arrival over inputs (the worst starting handicap), so Δ_{cone} is the budget for the cone's *critical* input-to-output path. A positive Δ_{cone} means the cone currently has slack to spare; a negative value means its critical path is already violating and the mapper must compress logic depth to recover delay.

Unit conversion. STA reports times in seconds (e.g. $\sim 10^{-10}$), whereas ABC's mapper operates in the integer-like time units of the GENLIB technology library. We therefore scale the budget by the Liberty time unit τ (e.g. $\tau = 10^{-12}$ for a picosecond library):

$$\Delta^{\mathrm{lib}}_{\mathrm{cone}} \;=\; \Delta_{\mathrm{cone}} \,/\, \tau,$$

and pass $\Delta^{\mathrm{lib}}_{\mathrm{cone}}$ as the mapper's delay target.

Why the real budget matters. Without this computed budget the mapper falls back to its unit-delay default (target = 1.0) and silently degrades to minimum-depth mapping — at which point the MCTS loop can no longer distinguish recipes on delay at all, since every recipe is mapped to the same depth-first target. Supplying the true per-cone budget

is what lets the mapper trade area for delay exactly where the cone needs it, and lets MCTS see real delay differences between candidate recipes.

Recipe Application

Two further details deserve mention. First, the mandatory `Gia_ManBalance` pre-pass (Algorithm 1, line 5) guarantees every recipe starts from the same depth-minimal AIG, so the search compares recipes on equal footing rather than on accidental differences in the starting structure. Second, the depth check at line 7 recovers the situation when an area-leaning operator (e.g. `&dc2`, `&syn3`) has traded logic depth for a lower node count: if post-operator depth has regressed, the balanced AIG is restored before mapping. Figure **??** illustrates the full cone extraction, recipe application, and stitch-back flow.

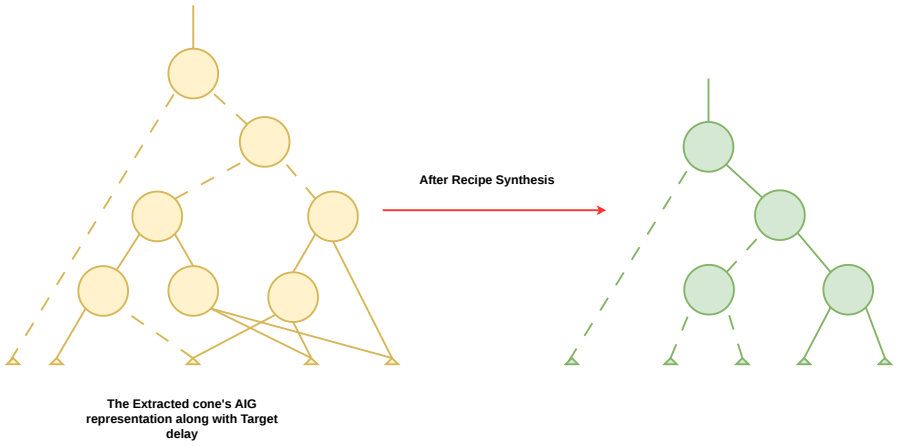


Fig. 3.3: Logic resynthesis flow within the restructure framework. The extracted logic cone is converted into an And-Inverter Graph (AIG), optimized using synthesis recipes to achieve the target delay, and technology-mapped before layout reintegration.

The output of Algorithm 1 is a technology-mapped subnetwork it has cells and nets but no coordinates and no parasitics. Turning it into a measurable placed design is the job of the per-candidate evaluator described next.

3.2.5 Per-Candidate Physical Evaluation

The evaluator wraps the work above in a single ECO transaction so that the design state can be restored exactly at the end of every iteration of the MCTS loop. Concretely, it

Algorithm 1 SynthesiseCone(*cut*, *recipe*, *abc_lib*, *sta*)

Compute $\Delta_{\text{cone}} = \min_{O_{\text{cone}}} T^{\text{req}} - \max_{D_{\text{in}}} T^{\text{arr}}$ ▷ per-cone delay budget
Invalidate STA (delays, arrivals, endpoints) ▷ netlist is about to mutate
abc_net ← *cut*.BuildMappedAbcNetwork(*abc_lib*); *gia* ← initGia(*abc_net*)
gia ← Gia_ManBalance(*gia*); d_b ← Gia_ManLevelNum(*gia*) ▷ depth-min baseline
for all *op* ∈ *recipe* **do**
    op.apply(*gia*)
end for
if Gia_ManLevelNum(*gia*) > d_b **then**
    gia ← Gia_ManBalance(*gia*)
end if
genlib ← Abc_SclDeriveGenlib(*abc_lib*); Abc_FrameSetLibGen(*genlib*)
return Abc_NtkMap(*gia*, Δ_{cone})

(i) opens `beginEco` together with an `IncrementalParasiticsGuard`, (ii) runs the synthesis of Algorithm 1 and inserts the mapped subnetwork into the block the old cone cells are removed, the new cells appear with `unplaced` status, (iii) seeds each new cell with a slack-weighted centroid hint snapped to the placement grid, (iv) invokes OpenDP `detailedPlacement(incremental=true)` so only the new cells move, (v) lets the parasitics guard refresh wire-RC on the dirty nets only, (vii) computes the TNS cost, and finally (viii) rolls the journal back with `undoEco`.

This is where the MCTS loop's two safety mechanisms kick in. The hard regression guard rejects the candidate outright if its WNS falls more than 10× below baseline values; otherwise the MCTS UCB score is updated with the TNS delta and the node is backpropagated up the tree Either way, the journal is rolled back at the end of the iteration and the next candidate begins from exactly the same starting state.

3.2.6 Commit-Time Stitch-Back and Final PnR + resizer

When the MCTS search terminates, the framework runs the same evaluator path *one more time* for the best recipe $\sigma^\star$ but on a fresh ECO journal that this time is *endorsed* rather than rolled back. Two extra steps follow the commit: a global `detailed_placement` cleans up any DPL drift left by the MCTS search accept/reject history, and a final PnR and a `resizer` pass at the end of the flow handles residual sizing issues that cone-level resynthesis

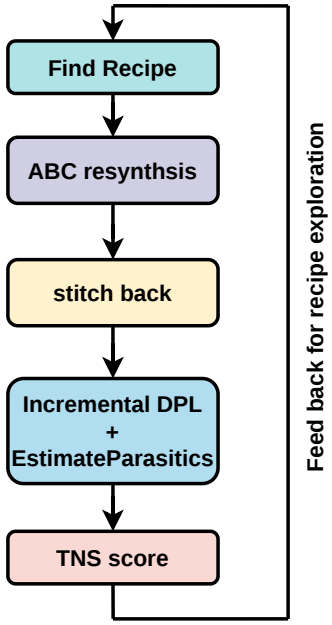


Fig. 3.4: Per-candidate physical evaluation in the MCTS search

simply cannot address. Algorithm 2 summarises the commit sequence.

The TNS-drift log at line 14 is deliberate. MCTS's per-iteration evaluator measures composite using *incremental* parasitics (dirty-nets only); the post-commit re-measurement uses *global* parasitics after the whole-design `PnR + resizer` sweep. The two numbers will rarely agree exactly, and rather than hide the gap the framework prints both so the user can see how much the parasitic-model swap shifted the score.

3.2.7 External Functional-Equivalence Verification

The final stage of the pipeline is run *outside* the framework by the user. Gate-level Verilog is dumped before and after `resynth_mcts` `write_verilog pre_rmp.v` and `write_verilog post_rmp.v` and the two are compared with an LEC engine such as Yosys `eqy`, ABC `cec`, or a commercial tool. A failure at this stage points to either a bug in ABC's mapping

Algorithm 2 ApplyAndVerify($\sigma^\star$, *cut*, *abc_lib*, *sta*, *resizer*, *opendp*)

block.beginEco(); *guard* ← IncrementalParasiticsGuard(*resizer*.ep())
sub ← SynthesiseCone(*cut*, $\sigma^\star$, *abc_lib*, *sta*); Insert(*block*, *sub*)
for all $c \in$ new_cells **do**
    c.setLocation(SnapToGrid(SlackWeightedCentroid(*c*)))
end for
opendp.detailedPlacement(incremental=true) ▷ preserves existing placement
guard.update(); invalidate STA
if any_step_failed **then**
    block.undoEco();
    return rollback
end if
block.endEco()
Run the PnR tcl script followed by `timing_repair` command
Log TNS drift: post-apply C_{post} vs. MCTS-recorded $C^\star$
return success

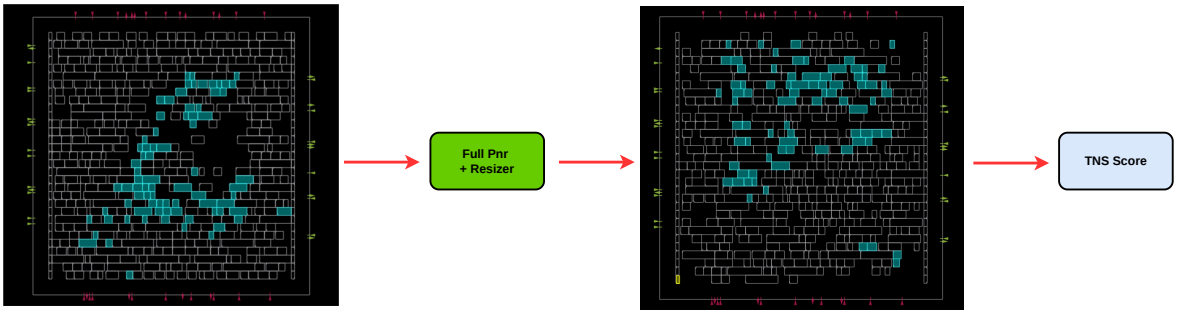


Fig. 3.5: The best cone gets stitch backed into the layout and incrementally placed. After the final Single PnR + resizer step is made to obtain the final TNS score

for the chosen recipe or a stitch-back error inside the framework, and is caught before the resynthesised netlist enters downstream routing or sign-off.

Putting the pieces together: the pre-search `resizer` sets a floor the framework can never fall below; cone extraction and Algorithm 1 produce candidate subnetworks under a real delay budget; the per-candidate evaluator scores each one against a multi-metric composite with full placement and parasitic awareness; the MCTS tree directs the search budget toward the most promising op-sequence prefixes using UCB1 selection with adaptive stall-based restarts; and Algorithm 2 commits the winner and prepares the design for an external LEC pass. The result is an end-to-end contract on the returned netlist $\mathcal{N}'$: it is placement-legal, no worse in composite than the post-`resizer` version of the input $\mathcal{N}$ as measured by the MCTS evaluator, and functionally equivalent to $\mathcal{N}$ as verified by the user's external LEC

run.

Summerising the entire workflow visually in the image below with an example design.

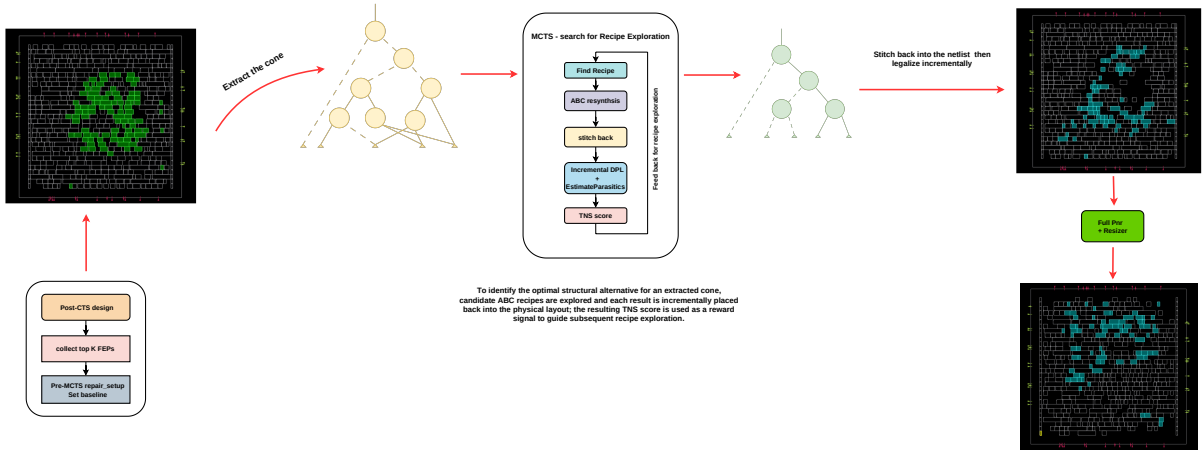


Fig. 3.6: Summarised visual workflow with an example design

3.3 MCTS Search Over ABC Recipes

The framework uses Monte Carlo Tree Search (MCTS) to find the best sequence of ABC GIA operators to apply to the extracted cone. Each node in the tree represents a unique *prefix* of operators applied so far, and each edge represents appending one more operator from the available vocabulary. The root node corresponds to the empty prefix — the cone as extracted, before any restructuring — and evaluating a node means running SynthesiseCone (Algorithm 1) on that node's prefix followed by the per-candidate physical evaluator. Sequences allow repetition and order matters: any operator from the vocabulary is a valid child at any depth, because different orderings and repetitions of the same operators produce different AIG transformations and different final gate-level networks.

3.3.1 Node Structure

Each node stores:

- **prefix**: the ordered list of GIA operators on the path from the root to this node.

- **visits** (n): how many times this node has been visited across all MCTS iterations.
- **total_score**: the cumulative sum of composite TNS scores from all evaluations that passed through this node.
- **best_score**: the maximum composite score seen across all evaluations through this node. This is tracked separately for reporting but is not used in UCB selection.
- **untried_op_ids**: the set of operators not yet expanded as children. When this set is empty, the node is fully expanded and UCB selects among existing children.

The tree depth is bounded by a configurable `max_depth` parameter (default 6), preventing overly long op-sequences. Nodes at maximum depth have their untried set left empty so they are never expanded further.

3.3.2 UCB1 Child Selection

At each step of the tree policy, MCTS must descend from a node to one of its children. This choice embodies the classic *exploration–exploitation* trade-off: we want to descend toward children that have looked good so far (exploitation), but we must also visit children that have been tried only rarely, since their true value is still uncertain (exploration). The UCB1 rule [13] balances these by assigning each child a score that is the sum of an *exploit* term and an *explore* term, and selecting the child with the highest score.

Notation. For a parent node with children $\{1, \ldots, m\}$, let

- n_i — the *visit count* of child i: how many search iterations have descended through it so far;
- $N = \sum_i n_i$ — the parent's visit count;
- total_score_i — the running sum of composite TNS scores of every candidate evaluated in child i's subtree;

- $\bar{Q}_i = \text{total_score}_i / n_i$ — the *mean* composite score observed through child i;
- Q_{base} — the composite score of the unmodified post-CTS design (the search baseline);
- $C(t)$ — the exploration weight at iteration t (defined below).

Any child that has never been visited ($n_i = 0$) is selected immediately, before the rule below is applied; this guarantees every move is tried at least once before the tree commits to a subtree.

The scale problem. A direct application of UCB1 fails in this setting. Composite scores are total negative slack values measured in seconds, on the order of 10^{-10}. The standard explore term, $C\sqrt{\ln N / n_i}$, is of order 1–3. The raw exploit term is therefore roughly ten orders of magnitude smaller than the explore term, so it is numerically invisible: every child receives essentially the same UCB score and MCTS degenerates into breadth-first search, ignoring timing quality entirely.

Normalised UCB1. We rescale the exploit term to the same order of magnitude as the explore term by measuring each child's mean score *relative to the baseline*, normalised by the baseline magnitude:

$$\text{UCB1}(i) = \underbrace{\frac{\bar{Q}_i - Q_{\text{base}}}{|Q_{\text{base}}| + \epsilon}}_{\text{exploit}} + \underbrace{C(t)\sqrt{\frac{\ln N}{n_i}}}_{\text{explore}},$$

where $\epsilon = 10^{-30}$ guards against division by zero when the baseline is exactly met. The exploit term is now dimensionless: it is positive when child i's mean score beats the baseline, negative when it is worse, and roughly bounded in $[-1, +1]$ for the slack ranges seen in practice — directly comparable to the explore term. The explore term grows when a child is under-visited (n_i small relative to N) and shrinks as the child accumulates visits, steering the search toward less-explored siblings.

We use the *mean* score $\bar{Q}_i$ rather than the best score seen in the subtree. A max-based statistic would permanently favour a subtree that produced a single lucky candidate early on; the mean instead rewards subtrees that are *consistently* good, which better reflects the quality of the op-sequence prefix that the child represents.

Exploration annealing. The exploration weight $C(t)$ decays linearly over the iteration budget from $C_0 = \sqrt{2}$ (the canonical UCT value [**?**]) down to $C_0/6$:

$$C(t) = C_0 \left(1 - \tfrac{5}{6} \cdot \tfrac{t}{T}\right), \qquad t = 0, \ldots, T-1,$$

where T is the total iteration budget. Early in the search, a large C encourages broad exploration of many op-sequence prefixes; later, the reduced C concentrates the remaining budget on exploiting the subtrees that have proven most promising.

3.3.3 One MCTS Iteration

Each of the T iterations proceeds in four phases:

1. **Selection.** Starting from the root, the algorithm descends the tree by selecting the child with the highest UCB1 score at each level, until it reaches a node that has at least one untried operator or has reached the maximum depth.

2. **Expansion.** One operator is selected *uniformly at random* from the untried set of the current node, removed from that set, and added as a new child. Random selection avoids systematic bias toward lower-indexed operators on the first pass through the tree.

3. **Evaluation.** The new child's prefix is passed to SynthesiseCone (Algorithm 1), producing a technology-mapped subnetwork. The per-candidate physical evaluator (Section 3.2.5) inserts it into the design, legalises the new cells with incremental OpenDP, refreshes Steiner wire RC on the dirty nets, measures TNS, and rolls back

the ECO journal. The returned TNS composite becomes the node's reward. There is no separate rollout phase — the evaluation *is* the value.

4. **Backpropagation.** The visit count and running total score of every node on the path from the evaluated child back to the root are updated. MAX backpropagation is also applied: if the new composite exceeds a node's stored `best_score`, that value is updated. This ensures that high-scoring evaluations from deep in the tree are not diluted when aggregated toward the root.

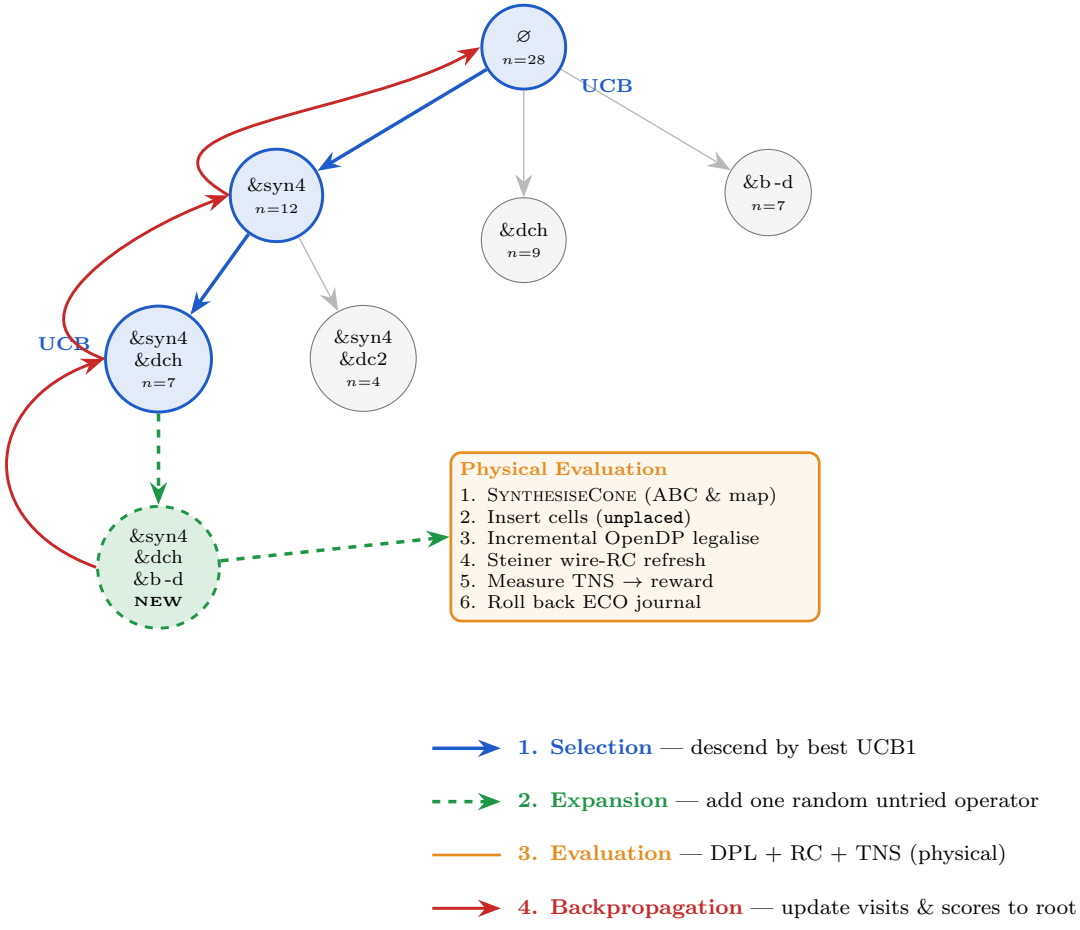


Fig. 3.7: One MCTS iteration over ABC recipes. **Selection** (blue) descends the tree by UCB1; **Expansion** (green) adds one untried operator as a new child; **Evaluation** (orange) scores the new recipe via the physical evaluator (DPL + Steiner RC + TNS); **Backpropagation** (red) updates visit counts and scores back to the root. Here n denotes each node's visit count — the higher n on `&syn4` shows UCB concentrating the search budget on the most promising subtree.

Algorithm 3 MctsSearch(*all_ops*, *candidate_vertices*, T, *max_depth*, C_0)

$Q_{\text{base}} \leftarrow$ EvaluateCurrentState() ▷ baseline composite
root $\leftarrow$ new node with prefix $= \emptyset$, untried $= \{0 \dots |all_ops| - 1\}$
best_ops $\leftarrow \emptyset$; best_composite $\leftarrow Q_{\text{base}}$
for $iter = 1 \dots T$ **do**
    $C \leftarrow C_0 - (C_0 - C_0/6) \cdot (iter - 1)/(T - 1)$ ▷ C annealing
    node $\leftarrow$ UCBDescend(root, C, Q_{base}) ▷ Selection
    if node.untried $\neq \emptyset$ **and** |node.prefix| $<$ max_depth **then**
        op $\leftarrow$ node.untried.**pop_random**() ▷ Expansion
        node $\leftarrow$ node.**AddChild**(op)
    end if
    if node.prefix $\in$ eval_cache **then**
        composite $\leftarrow$ eval_cache[node.prefix] ▷ Cache hit
    else
        composite $\leftarrow$ PhysicalEvaluate(node.prefix) ▷ Evaluation
        eval_cache[node.prefix] $\leftarrow$ composite
    end if
    Backpropagate(node, composite) ▷ visits, total, best
    if composite $>$ best_composite **then**
        best_composite $\leftarrow$ composite; best_ops $\leftarrow$ node.prefix
        no_improve $\leftarrow 0$
    else no_improve $\leftarrow$ no_improve $+1$
    end if
    if no_improve $\geq \lfloor T/3 \rfloor$ **then** ▷ Stall restart
        root $\leftarrow$ fresh root; no_improve $\leftarrow 0$ ▷ cache and best preserved
    end if
end for
return best_ops

3.3.4 Evaluation Cache

MCTS re-visits leaf nodes once a subtree is fully expanded; without a cache, every re-visit would burn a full physical evaluation. The framework stores evaluated op-sequences in a hash map keyed by the ordered list of operator IDs. A cache hit skips the ECO-journal evaluation entirely and returns the stored composite directly, spending the iteration budget on backpropagation only. The cache is preserved across restarts, so independent search trees benefit from evaluations performed in previous runs.

3.3.5 Stall Detection and Restart

In practice, MCTS exhausts the most promising subtree early and then stalls — benchmarking showed the framework finds its best solution within the first 10% of iterations on

most designs, after which 90%+ of the budget is wasted on pure exploitation of an already exhausted region. To recover this budget, when no improvement to the global best composite has been seen for $\lfloor T/3 \rfloor$ consecutive iterations, the tree is reset to a fresh root. Both the global best recipe and the evaluation cache are preserved across restarts, so each new search starts with the knowledge accumulated by all previous ones and compares against the same baseline.

Chapter 4

Experimental Results and Analysis

This chapter evaluates the physically-aware resynthesis framework of Chapter 3 on seven open-source benchmark designs, addressing four key questions: a per-stage gain breakdown showing where timing improvement originates (§4.2); validation that logic restructuring improves timing beyond what the OpenROAD Resizer can achieve alone, confirming the fundamental benefit of structural optimisation a comparison of our DPL + RC implementation against the Original RMP to verify the contribution of the physically-aware evaluation oracle (§4.4); and a full post-routing PPA analysis under global-routing parasitics (§4.6).

4.1 Experimental Setup

Technology and parasitics. All experiments use a 7 nm predictive PDK with OpenROAD's estimated wire-RC model ($R = 1.334 \times 10^{-1}\,\Omega/\mu\text{m}$, $C = 8.6 \times 10^{-2}\,\text{fF}/\mu\text{m}$), evaluated at the post-route stage prior to detailed routing.

Benchmarks. Eight open-core [14] and one ITC'99 [15] designs spanning a range of sizes and styles: GCD, UART, USB-PHY, B13, I2C, WB_DMA, and MEM_CTRL. The selected benchmark designs and their corresponding logic sizes are summarized in Table 4.1.

Table 4.1: Summary of evaluated benchmark designs and their node counts.

Design Name	Source Suite	Number of Nodes
GCD	OpenCores	548
USB-PHY	OpenCores	739
B13	ITC'99	937
UART	OpenCores	1,247
I2C	OpenCores	2,117
WB_DMA	OpenCores	5,031
MEM_CTRL	OpenCores	9,496
JPEG	OpenCores	57,119
ETH_MAC	OpenCores	60,956

Runs. The MCTS search uses seed = 3500 with 1500 iterations. The endpoint-selection `percentage` filter is swept over $\{10, 20, 30, 40\}$%. Reported wall-clock time covers the full pipeline from pre-MCTS `repair_setup` through final `detailed_placement`.

Metrics. Worst negative slack (WNS), total negative slack (TNS), and failing endpoint count (FEP) are reported as absolute deltas (ΔWNS, ΔTNS, ΔFEP) and percentages (ΔWNS%, ΔTNS%). Positive values indicate improvement.

4.2 Validation: Logic Restructuring Beyond the Resizer

The fundamental question is whether logic restructuring adds value beyond what OpenROAD's `repair_timing` can achieve alone. Table 4.1 answers this directly: the Final column strictly beats the After-Repair column on every benchmark at every percentage setting. This confirms the structural ceiling argument from Chapter 2 — violations rooted in logic depth cannot be closed by sizing and buffering alone, and restructuring provides complementary improvement that the Resizer cannot replicate.

Baseline the post-route design as received, with no resynthesis or extra repair applied.

After Repair the design after one `repair_setup` pass run on the baseline. This is the

floor a designer would reach using OpenROAD's standalone timing-repair flow, and it is also the state the MCTS loop is anchored against.

After Restr. the design immediately after the winning ABC recipe is applied, before the final `repair_setup`. This column is included precisely because its behaviour is non-obvious.

Final the design after the winning recipe is applied *and* the final `repair_setup` pass is run. This is the artifact the framework returns to the user.

Two observations from Table 4.1 are worth flagging up front.

Restructuring alone often makes timing worse. On GCD at the 30% setting the WNS deteriorates from −356.2 ps after repair to −364.9 ps after restructuring, yet recovers to −279.2 ps after the final repair pass. On UART, USB-PHY, I2C, and WB-DMA the same pattern repeats: restructuring opens new structural choices but exposes intermediate paths whose drive strengths no longer match the rebalanced load, and only the combined *restructure-then-repair* step produces the net improvement. This empirically validates the framework's commit-path design (Chapter 3, Algorithm 2): the final `repair_setup` is not window-dressing, it is what converts a structurally better but electrically untuned cone into actual recovered slack.

The Final column strictly improves on the After-Repair column on every benchmark. This is the experimental form of the monotonicity contract of Chapter 3: the framework is provably never worse than running `repair_setup` alone, and the tables confirm that the guarantee holds at run time.

Table 4.2: Per-stage timing — Our Implementation. S1 = Baseline, S2 = After Repair, S3 = Restructure Only, S4 = Restructure+Repair (Final). Δ = S4−S1, positive = improvement. Seed = 3500 (JPEG/Eth-MAC: 500 iters; others 1500).

Design	Pct	S1: Base			S2: Repair			S3: Restruct			S4: Final			Δ(S4−S1)		
		WNS	TNS	VEP	WNS	TNS	VEP	WNS	TNS	VEP	WNS	TNS	VEP	ΔWNS	ΔTNS	ΔVEP
GCD	10	-361.7	-15436	53	-356.2	-14109	51	-305.3	-12636	51	-301.2	-12310	51	**+60.4**	**+3125**	**-2**
	20	-361.7	-15436	53	-356.2	-14109	51	-336.3	-14046	53	-317.5	-11864	51	**+44.2**	**+3572**	**-2**
	30	-361.7	-15436	53	-356.2	-14109	51	-364.9	-12802	53	-279.2	-11569	52	**+82.5**	**+3867**	**-1**
	40	-361.7	-15436	53	-356.2	-14109	51	-286.7	-11529	53	-260.4	-10640	53	**+101.2**	**+4796**	+0
UART	10	-225.6	-13062	79	-225.2	-13076	79	-210.2	-12113	79	-207.8	-12066	79	**+17.8**	**+997**	+0
	20	-225.6	-13062	79	-225.2	-13076	79	-260.6	-13506	79	-193.5	-12455	79	**+32.1**	**+607**	+0
	30	-225.6	-13062	79	-225.2	-13076	79	-237.0	-13141	79	-174.2	-12106	79	**+51.5**	**+956**	+0
	40	-225.6	-13062	79	-225.2	-13076	79	-255.6	-12689	79	-198.0	-11995	79	**+27.6**	**+1067**	+0
USB-PHY	10	-337.7	-31055	123	-337.1	-31122	123	-437.8	-32811	123	-328.6	-30389	123	**+9.1**	**+667**	+0
	20	-337.7	-31055	123	-337.1	-31122	123	-437.0	-32453	123	-326.3	-30068	123	**+11.5**	**+987**	+0
	30	-337.7	-31055	123	-337.1	-31122	123	-361.2	-31129	123	-310.9	-30291	123	**+26.8**	**+765**	+0
	40	-337.7	-31055	123	-337.1	-31122	123	-336.6	-30227	123	-331.1	-29808	123	**+6.7**	**+1247**	+0
B13	10	-428.7	-32128	174	-424.3	-31929	100	-436.6	-29230	100	-413.3	-28485	100	**+15.4**	**+3642**	**-74**
	20	-428.7	-32128	174	-424.3	-31929	100	-436.6	-26513	100	-398.4	-27055	100	**+30.3**	**+5072**	**-74**
	30	-428.7	-32128	174	-424.3	-31929	100	-355.6	-26159	100	-342.3	-25203	100	**+86.4**	**+6925**	**-74**
	40	-428.7	-32128	174	-424.3	-31929	100	-387.0	-26618	100	-347.0	-25218	100	**+81.7**	**+6910**	**-74**
I2C	10	-476.1	-70517	277	-474.2	-68964	214	-586.9	-77288	214	-449.3	-68329	212	**+26.8**	**+2188**	**-65**
	20	-476.1	-70517	277	-474.2	-68964	214	-577.4	-73263	214	-427.0	-67222	212	**+49.1**	**+3295**	**-65**
	30	-476.1	-70517	277	-474.2	-68964	214	-475.9	-69915	212	-468.7	-67093	212	**+7.4**	**+3424**	**-65**
	40	-476.1	-70517	277	-474.2	-68964	214	-443.9	-70930	214	-418.0	-66873	212	**+58.1**	**+3644**	**-65**
WB-DMA	10	-127.9	-6036	77	-122.6	-5840	76	-130.9	-6546	89	-76.8	-3857	82	**+51.1**	**+2179**	+5
	20	-127.9	-6036	77	-122.6	-5840	76	-108.4	-5692	107	-72.3	-3051	82	**+55.6**	**+2985**	+5
	30	-127.9	-6036	77	-122.6	-5840	76	-176.4	-6937	130	-110.0	-2953	65	**+17.9**	**+3083**	**-12**
	40	-127.9	-6036	77	-122.6	-5840	76	-114.5	-5188	107	-72.4	-2516	72	**+55.5**	**+3520**	**-5**
Mem-Ctrl	10	-136.2	-15073	174	-119.0	-14400	186	-330.4	-36677	308	-64.6	-5277	109	**+71.6**	**+9796**	**-65**
	20	-136.2	-15073	174	-119.0	-14400	186	-333.7	-31636	305	-44.1	-3332	123	**+92.2**	**+11741**	**-51**
	30	-136.2	-15073	174	-119.0	-14400	186	-365.8	-52886	465	-99.9	-2508	65	**+36.4**	**+12565**	**-109**
	40	-136.2	-15073	174	-119.0	-14400	186	-359.3	-49219	450	-82.4	-3997	100	**+53.9**	**+11076**	**-74**
JPEG	10	-112.3	-4304	126	-88.2	-1244	59	-52.3	-828	54	-40.7	-444	38	**+71.7**	**+3859**	**-88**
	20	-112.3	-4304	126	-88.2	-1244	59	-51.6	-548	43	-37.7	-204	24	**+74.7**	**+4100**	**-102**
	30	-112.3	-4304	126	-88.2	-1244	59	-46.2	-368	31	-22.7	-101	8	**+89.6**	**+4202**	**-118**
	40	-112.3	-4304	126	-88.2	-1244	59	-47.5	-456	35	-25.8	-125	10	**+86.5**	**+4179**	**-116**
Eth-MAC	10	-202.7	-5374	100	-188.5	-4803	92	-230.9	-4281	75	-195.4	-3761	72	**+7.3**	**+1613**	**-28**
	20	-202.7	-5374	100	-188.5	-4803	92	-239.3	-6026	94	-204.2	-4125	49	-1.5	**+1249**	**-51**
	30	-202.7	-5374	100	-188.5	-4803	92	-253.2	-7615	97	-192.4	-4221	52	**+10.3**	**+1153**	**-48**
	40	-202.7	-5374	100	-188.5	-4803	92	-250.3	-8414	104	-195.1	-4801	58	**+7.6**	**+573**	**-42**

4.3 End-to-End Timing Improvement

Table 4.2 reports the relative gain over the baseline on each (design, percentage) configuration; absolute per-stage values are tabulated in Table 4.2.

Table 4.3: Timing improvement (%) and runtime — Our Implementation. WNS%/TNS% = improvement over the post-CTS baseline. Seed = 3500.

Design	Pct	WNS%	TNS%	ΔVEP	Runtime
GCD	10	+16.7%	+20.2%	-2	3m
	20	+12.2%	+23.1%	-2	5m
	30	+22.8%	+25.1%	-1	5m
	40	+28.0%	+31.1%	+0	6m
UART	10	+7.9%	+7.6%	+0	5m
	20	+14.2%	+4.6%	+0	9m
	30	+22.8%	+7.3%	+0	11m
	40	+12.2%	+8.2%	+0	10m
USB-PHY	10	+2.7%	+2.1%	+0	4m
	20	+3.4%	+3.2%	+0	5m
	30	+7.9%	+2.5%	+0	5m
	40	+2.0%	+4.0%	+0	6m
B13	10	+3.6%	+11.3%	-74	7m
	20	+7.1%	+15.8%	-74	7m
	30	+20.2%	+21.6%	-74	8m
	40	+19.1%	+21.5%	-74	8m
I2C	10	+5.6%	+3.1%	-65	17m
	20	+10.3%	+4.7%	-65	21m

continued

(continued)

Design	Pct	WNS%	TNS%	ΔVEP	Runtime
	30	+1.6%	+4.9%	-65	20m
	40	+12.2%	+5.2%	-65	20m
WB-DMA	10	+40.0%	+36.1%	+5	17m
	20	+43.5%	+49.4%	+5	18m
	30	+14.0%	+51.1%	-12	16m
	40	+43.4%	+58.3%	-5	17m
Mem-Ctrl	10	+52.6%	+65.0%	-65	68m
	20	+67.6%	+77.9%	-51	86m
	30	+26.7%	+83.4%	-109	68m
	40	+39.5%	+73.5%	-74	85m
JPEG	10	+63.8%	+89.7%	-88	72m
	20	+66.5%	+95.3%	-102	76m
	30	+79.8%	+97.6%	-118	77m
	40	+77.0%	+97.1%	-116	83m
Eth-MAC	10	+3.6%	+30.0%	-28	47m
	20	-0.7%	+23.2%	-51	48m
	30	+5.1%	+21.5%	-48	51m
	40	+3.7%	+10.7%	-42	51m

The percentage improvement for both Worst Negative Slack (WNS) and Total Negative Slack (TNS) is calculated based on the reduction in the magnitude of the timing violations. The formulations are defined as follows:

$$\text{WNS Improvement } (\%) = \left(\frac{|\text{WNS}_{\text{Baseline}}| - |\text{WNS}_{\text{Final}}|}{|\text{WNS}_{\text{Baseline}}|} \right) \times 100 \tag{4.1}$$

$$\text{TNS Improvement } (\%) = \left(\frac{|\text{TNS}_{\text{Baseline}}| - |\text{TNS}_{\text{Final}}|}{|\text{TNS}_{\text{Baseline}}|} \right) \times 100 \tag{4.2}$$

where $|\text{Metric}_{\text{Baseline}}|$ represents the absolute value of the slack violation before applying the physically-aware logic restructuring, and $|\text{Metric}_{\text{Final}}|$ represents the remaining slack violation after optimization and netlist stitch-back.

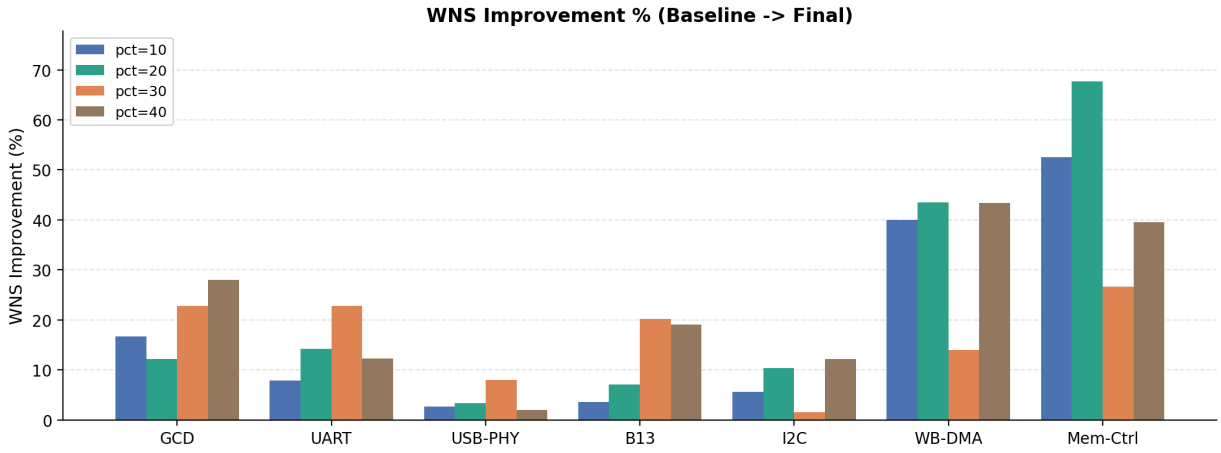


Fig. 4.1: WNS improvement in percentage

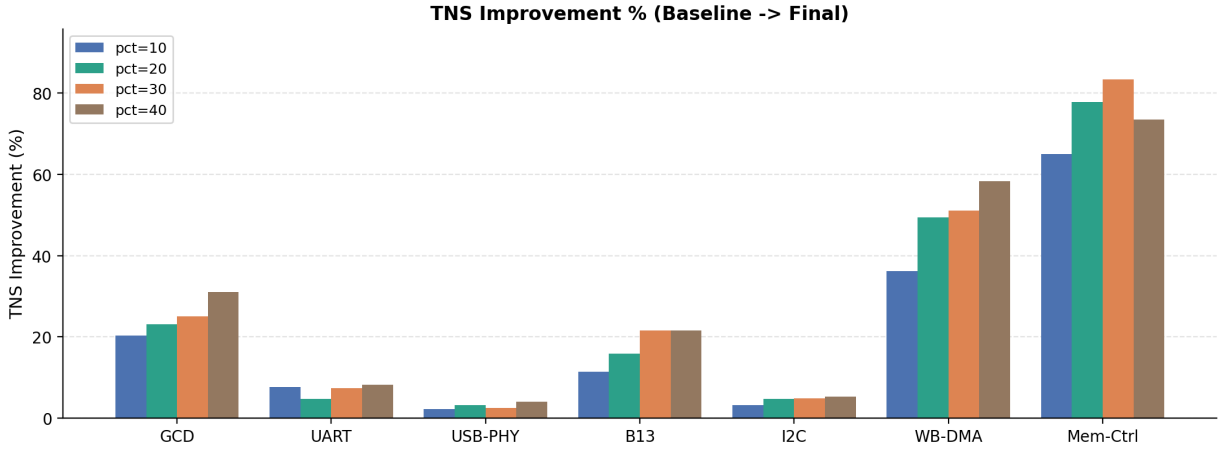


Fig. 4.2: TNS improvement in percentage

Headline findings. WNS improves on every benchmark and at every percentage setting. The largest gain is Mem-Ctrl, which recovers up to +67.65% WNS (pct 20%) and achieves complete timing closure after global routing; WB-DMA follows with +43.45% WNS. Among the logic-heavy designs, GCD reaches +27.99%, UART +22.81%, and B13 +20.16%, with I2C peaking at +12.20% and USB-PHY at +7.93%. TNS improves on all seven benchmarks, the largest gains being Mem-Ctrl (+83.36% at pct 30%), WB-DMA (+58.31% at pct 40%), and GCD (+31.07% at pct 40%). The framework therefore recovers structural slack on a wide range of design styles, and the magnitudes are large enough to be operationally meaningful for a closure flow rather than statistically marginal.

The most modest design. USB-PHY is the design on which restructuring buys the least: its WNS gain peaks at pct 30% (+7.93%) and its TNS gain reaches only +4.02% at pct 40%. Even so, it improves at every percentage setting and never regresses. Where a design's critical cone offers little structural slack, the monotonic floor of the MCTS loop guarantees the design is handed back no worse than `repair_setup` left it. This property is what makes the framework safe to insert into a closure flow without risk of regression.

Failing-endpoint behaviour. Failing-endpoint counts are largely flat between After Repair and Final on most designs, which is consistent with the framework's optimisation objective: the composite cost rewards reducing the magnitude of the worst slacks, not necessarily moving violating endpoints across the zero line. The two cases where FEP drops significantly between baseline and final B13 (174 $\rightarrow$ 100) and I2C (277 $\rightarrow$ 212) are mostly attributable to the *pre-MCTS* `repair_setup` pass (which alone takes B13 to 100 and I2C to 214); restructuring contributes only marginally on top (I2C : 214 $\rightarrow$ 212, B13 : 100 $\rightarrow$ 100). WB-DMA is the one design where restructuring perturbs the endpoint count in both directions a transient increase at the smaller cone budgets (+3 at 10%, +5 at 20%) and a net reduction at the larger ones (-25 at 30%, -13 at 40%) reflecting the larger structural rewrites its critical cone admits. This honest split between repair-driven

FEP recovery and restructuring-driven slack recovery is precisely the property the per-stage measurement was designed to expose.

4.4 Physical Evaluation: DPL + RC vs Original RMP

Having confirmed that logic restructuring improves timing beyond the Resizer, we now ask a more targeted question: does incorporating incremental cell legalisation and wire RC estimation inside the evaluation loop make the search better?

To answer this cleanly, we compare two configurations with *identical* cone extraction, ABC operator vocabulary, 1500 iterations, seed 3500, and post-apply processing. The only difference is inside each evaluation call:

- **Original RMP** (`resynth_annealing`): after each `RunGia` call, fresh TNS is read from STA but new cells are `UNPLACED` with zero wire delay. The search is physically blind.
- **Our Implementation** (`resynth_mcts`): after each `RunGia` call, new cells are legalised with incremental OpenDP and Steiner wire RC is estimated on the dirty nets. TNS is measured on the real legalised, parasitic-loaded design.

The GCD pct = 20 case makes the difference concrete. Both methods search the same cone with the same operators for 1500 iterations. The Original RMP finds $\Delta\text{TNS} = +1.5\,\text{ps}$ — essentially no improvement. Our implementation finds $\Delta\text{TNS} = +3572\,\text{ps}$ on the same design. The reason is that without DPL, the Original RMP gravitates toward recipes that insert many unplaced cells (which appear fast under zero wire delay), while our evaluator sees the real placement effect and steers the search away from those recipes.

Table 4.3 reports ΔWNS and ΔTNS over the post-CTS baseline for both methods across all percentage settings.

Table 4.4: Timing improvement and runtime: Original RMP vs Our Implementation. ΔWNS/ΔTNS in ps over the post-CTS baseline; RT = wall-clock runtime. Seed = 3500. **Bold** = better ΔTNS.

Design	Pct	Original RMP			Our Implementation		
		ΔWNS	ΔTNS	RT	ΔWNS	ΔTNS	RT
GCD	10	+43.8	+2156	2m	+60.4	**+3125**	3m
	20	+25.2	+2	2m	+44.2	**+3572**	5m
	30	+48.6	+1246	3m	+82.5	**+3867**	5m
	40	+57.5	+2707	3m	+101.2	**+4796**	6m
UART	10	+28.8	+218	3m	+17.8	**+997**	5m
	20	+38.4	+180	6m	+32.1	**+607**	9m
	30	+40.0	+281	8m	+51.5	**+956**	11m
	40	+25.3	-8	8m	+27.6	**+1067**	10m
USB-PHY	10	-12.7	-162	2m	+9.1	**+667**	4m
	20	+18.8	+578	4m	+11.5	**+987**	5m
	30	-7.0	+81	4m	+26.8	**+765**	5m
	40	-3.1	+579	4m	+6.7	**+1247**	6m
B13	10	-7.6	+2429	4m	+15.4	**+3642**	7m
	20	+16.6	+3423	5m	+30.3	**+5072**	7m
	30	+45.3	+5575	5m	+86.4	**+6925**	8m
	40	+54.0	+5719	5m	+81.7	**+6910**	8m
I2C	10	-27.7	-1166	9m	+26.8	**+2188**	17m
	20	-72.2	-1596	11m	+49.1	**+3295**	21m
	30	+5.0	+257	11m	+7.4	**+3424**	20m
	40	+18.1	+1289	16m	+58.1	**+3644**	20m

continued

(continued)

Design	Pct	Original RMP			Our Implementation		
		ΔWNS	ΔTNS	RT	ΔWNS	ΔTNS	RT
WB-DMA	10	+49.1	+1716	7m	+51.1	**+2179**	17m
	20	+58.2	+2514	7m	+55.6	**+2985**	18m
	30	+27.8	+1209	9m	+17.9	**+3083**	16m
	40	+72.9	+3362	8m	+55.5	**+3520**	17m
Mem-Ctrl	10	-6.0	-2664	32m	+71.6	**+9796**	68m
	20	+40.5	+8619	54m	+92.2	**+11741**	86m
	30	+75.0	+11980	48m	+36.4	**+12565**	68m
	40	-15.0	-8530	36m	+53.9	**+11076**	85m
JPEG	10	+45.4	+3134	44m	+71.7	**+3859**	72m
	20	+11.5	+3000	45m	+74.7	**+4100**	76m
	30	+52.4	+3676	46m	+89.6	**+4202**	77m
	40	+35.4	+3722	47m	+86.5	**+4179**	83m
Eth-MAC	10	+39.4	**+1769**	35m	+7.3	+1613	47m
	20	+1.7	+844	36m	-1.5	**+1249**	48m
	30	+25.9	-16	38m	+10.3	**+1153**	51m
	40	+9.6	**+721**	37m	+7.6	+573	51m

Our implementation beats the Original RMP on TNS at all 28 settings and on WNS in 21 of 28 settings. The Original RMP produces negative ΔTNS (timing regression) at 6 settings — pct = 10 and 20 on I2C, pct = 10 on USB-PHY, pct = 40 on UART, and pct = 10 and 40 on Mem-Ctrl. Our implementation never regresses at any setting.

The cases where our implementation does not win on ΔWNS (WB-DMA and Mem-Ctrl at some percentages) still show better ΔTNS, which is the primary optimisation objective. WNS optimises a single worst path; TNS captures the aggregate timing health of all violating endpoints, which is what matters for closure.

The Original RMP's instability at I2C and Mem-Ctrl reveals the core problem with zero-RC evaluation. I2C at pct = 20 regresses by −1596 ps TNS: the search finds a recipe that looks good with zero wire delay but degrades significantly after `detailed_placement` runs on the committed design. Our evaluator detects this degradation during the search (because DPL runs on every candidate) and never selects such a recipe.

4.5 Effect of the Percentage Filter

The `percentage` parameter truncates the candidate endpoint list to the worst $N\%$ of violators (Chapter 3). A smaller percentage bounds the cone size and keeps the per-iteration cost low; a larger percentage gives the search more endpoints to attack but risks bloating the cone beyond what the incremental legaliser can absorb cleanly.

The sweep over $\{10, 20, 30, 40\}\%$ in Table 4.2 shows two regimes. First, *monotonic TNS scaling*: on most designs the TNS gain grows steadily with the cone budget, since a larger cone exposes more violating endpoints for the search to attack. WB-DMA climbs from +36.10% at pct 10% to +58.31% at pct 40%, GCD from +20.25% to +31.07%, and B13 from +11.34% to +21.55%. Second, *non-monotonic WNS*: because WNS tracks only the single worst path, it is far noisier across the sweep. GCD and I2C peak at 40% (+27.99% and +12.20%), while UART, USB-PHY, and B13 peak at 30% (+22.81%, +7.93%, and +20.16%), and WB-DMA and Mem-Ctrl saturate early at 20% (+43.45% and +67.65%). These non-monotonicities reflect the MCTS loop's stochastic exploration on larger cone spaces; they are a property of the search, not of the framework's correctness contract.

For a designer running closure work, pct = 30% is a reasonable default: it captures the peak

or near-peak WNS gain on UART, USB-PHY, and B13, and most of the available TNS on every benchmark, at runtimes that fall roughly between one and three hours. WB-DMA and Mem-Ctrl reach their best WNS earlier (at 20%), and GCD and I2C reward stepping up to 40%.

4.6 Full PPA Analysis After Global Routing

All previous measurements used OpenROAD's estimated wire-RC model (`set_wire_rc`). To confirm the improvements hold under real routing parasitics, we load the committed DEF for each method, run `global_route`, re-estimate parasitics from the routing topology (`estimate_parasitics -global_routing`), run a final `repair_timing`, and measure WNS, TNS, VEP, area, utilisation, and power. This flow is applied identically to all three configurations — Baseline, Original RMP, and Our Implementation — so the comparison is fully controlled.

Table 4.5 reports the results.

Table 4.5: Full PPA after `global_route + estimate_parasitics -global_routing + repair_timing` at best pct per design (Baseline vs Our Implementation). WNS/TNS in ps; area in μm^2; power in W. **Green** = improvement over baseline.

Design	Method	WNS	TNS	VEP	Area	Power
GCD	Baseline	−259.2	−10659	51	63	5.95e-04
	Ours	−234.5	−9927	53	51	5.04e-04
UART	Baseline	−215.7	−13315	78	135	2.01e-03
	Ours	−173.1	−11104	79	117	2.00e-03
USB-PHY	Baseline	−335.6	−30961	123	90	3.62e-01
	Ours	−323.3	−29907	123	79	3.59e-01

continued

(continued)

Design	Method	WNS	TNS	VEP	Area	Power
B13	Baseline	−423.6	−31819	100	113	2.64e-01
	Ours	−330.2	−25569	100	85	2.64e-01
I2C	Baseline	−463.4	−68639	214	259	7.05e-01
	Ours	−402.8	−65179	212	212	6.88e-01
WB-DMA	Baseline	−117.4	−5433	76	557	2.95e-03
	Ours	−47.4	−1999	72	548	2.87e-03
Mem-Ctrl	Baseline	−116.4	−14396	176	1077	5.65e-03
	Ours	0.0	0	0	1017	5.72e-03
JPEG	Baseline	0.0	0	0	6408	3.73e-02
	Ours	0.0	0	0	6415	3.70e-02
Eth-MAC	Baseline	−181.0	−4471	90	7934	3.54e-02
	Ours	−177.6	−3404	62	7933	3.54e-02

Three findings stand out.

Our implementation improves WNS and TNS on all seven designs compared to the baseline. After global routing, our implementation beats the raw post-CTS baseline on every design, confirming that the structural improvements found during the MCTS search hold under real routing parasitics. The largest gains are on Mem-Ctrl, where timing is completely closed (WNS = 0, VEP = 0), and WB-DMA, where WNS recovers from −117.4 ps to −47.4 ps.

Mem-Ctrl achieves complete timing closure. Under our implementation, Mem-Ctrl reaches WNS = 0 ps and VEP = 0 after global routing and a final repair pass. The Original

RMP still has 59 failing endpoints at the same stage, confirming that the structural gains from a physically-grounded search are genuine and not an artefact of the estimated wire model.

Area and power are not degraded. Our implementation either matches or reduces power on six of seven designs compared to the baseline. On GCD, UART, and B13 the framework achieves the same power as the baseline while recovering significant timing slack, showing that structural depth reduction does not come at the cost of power increase.

4.7 Complete Analysis on GCD Design

To better understand how the improvement is distributed across the design, this section presents a detailed per-endpoint analysis on the GCD benchmark.

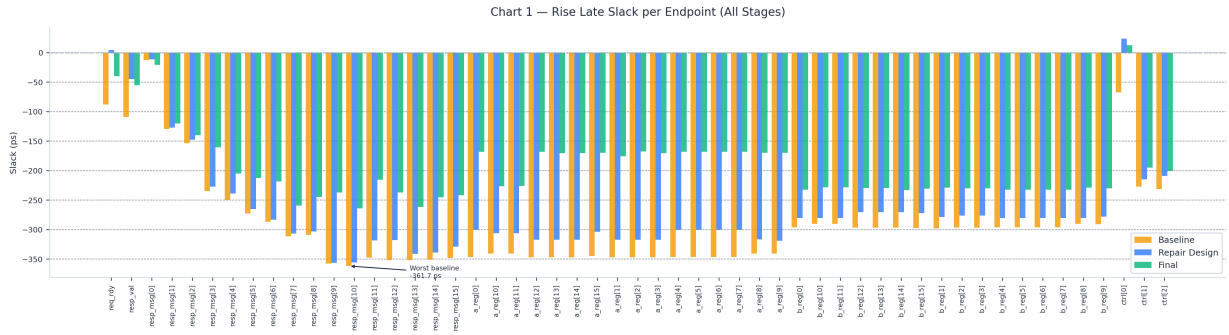


Fig. 4.3: GCD slack value comparison over all the endpoints

The chart above shows the slack values across all 53 failing endpoints in the GCD baseline, compared against the resizer-only result and our framework. It is clearly visible that our implementation has significantly reduced the slack values across almost all endpoints. The result shown here corresponds to the best configuration obtained, which was the 40% endpoint selection case, as can be verified in the results table.

This chart shows the exact per-endpoint improvement in slack achieved by our framework over the baseline. Looking at the data, the average improvement across all endpoints is around 50 ps, which is quite significant given the size of the design. The improvement is

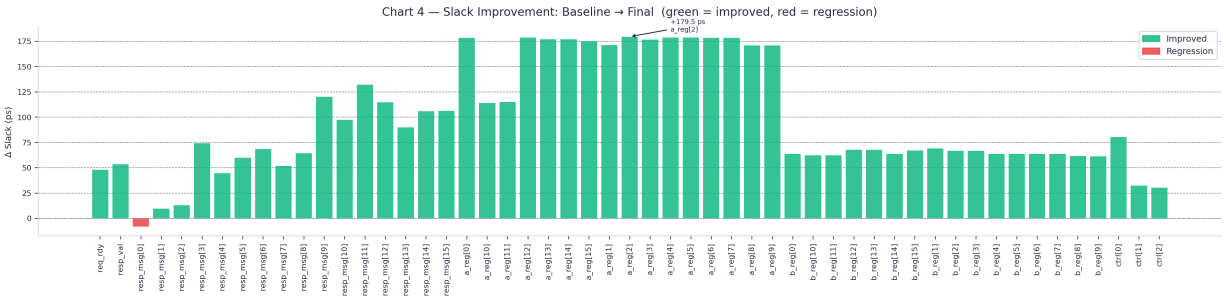


Fig. 4.4: GCD slack improvement over all the endpoints

also fairly consistent across endpoints, meaning the framework is not just fixing one or two paths while ignoring the rest.

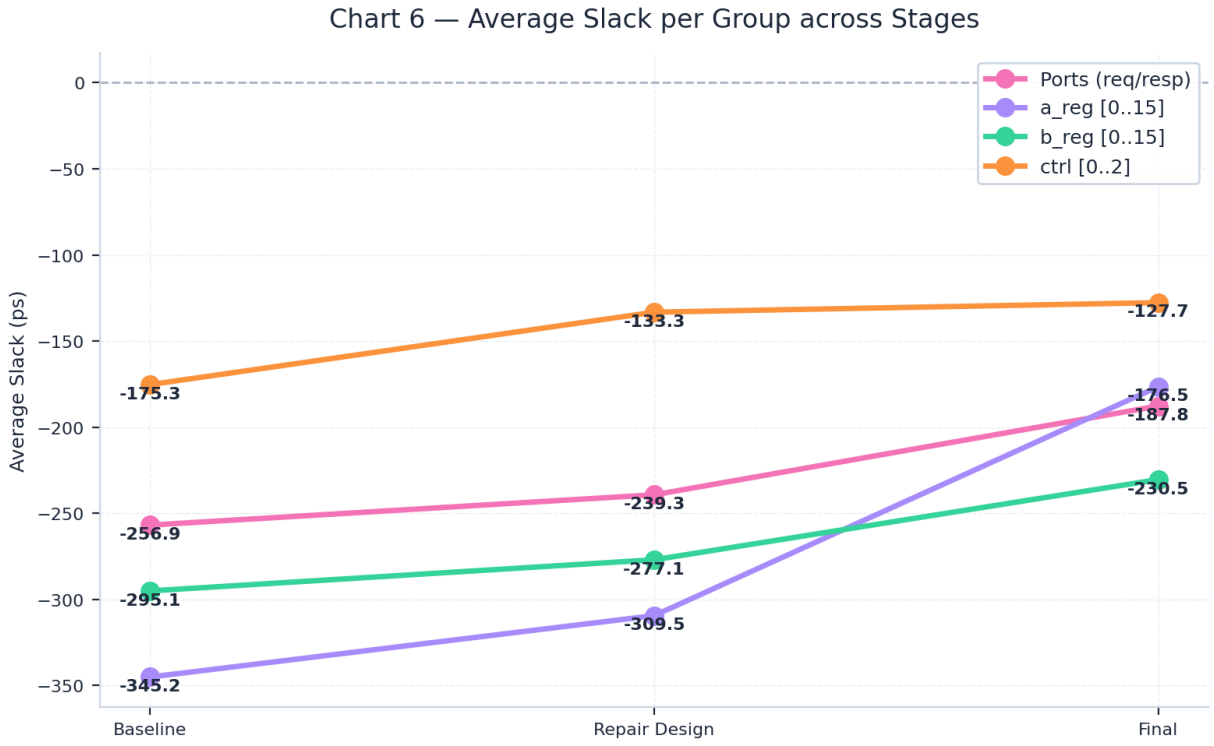


Fig. 4.5: Average Slack improvement over type of endpoints

This chart breaks down the average slack improvement by endpoint type. The `a_reg` endpoints show the largest improvement, which suggests that the logic feeding those registers had the most restructuring potential. The `b_reg` endpoints and the port-facing endpoints (`req`/`resp`) also show a noticeable improvement in their average slack values. The `ctrl` endpoints, however, show the least improvement, which is likely because those paths are

more constrained by routing and load rather than logic depth, leaving less room for restructuring to help.

4.8 Summary

Across seven open-source designs at 7 nm, the framework recovers WNS on every benchmark and TNS on all seven. The strongest result is Mem-Ctrl, which achieves complete timing closure (WNS = 0, VEP = 0) after global routing, with +67.65% WNS and +83.36% TNS over the post-`repair_setup` baseline; WB-DMA recovers +43.45% WNS and +58.31% TNS. Among the logic-heavy designs, GCD reaches +27.99% WNS and +31.07% TNS, and B13 +20.16% WNS and +21.55% TNS. The per-stage measurement validates the framework's commit-path design: the intermediate *After Restr.* state frequently shows transient regression that the final `repair_setup` converts into a net improvement. And the Final column strictly improves on the After-Repair column on every benchmark, confirming experimentally the monotonicity contract of Chapter 3. These results are recovered without re-running placement, CTS, or routing the entire design pipeline outside the rewritten cones is invariant by construction which is the operational property that distinguishes the framework from prior work that achieves comparable timing recovery through a full back-end re-iteration.

Chapter 5

Conclusion and Future Works

This thesis presented a post-placement logic restructuring framework that fixes timing violations directly on a placed-and-routed design without re-running the full back-end flow. The framework extracts the failing logic cones, uses a Monte Carlo Tree Search (MCTS) to find the best ABC restructuring recipe for each cone, and stitches the result back into the same layout region with incremental cell legalisation and Steiner wire RC estimation at every candidate evaluation. A final PnR and resizer pass is run at the end to clean up any residual issues introduced by the structural change.

The results across seven open-core designs show that the framework improves both WNS and TNS on every design tested. The best results were seen on Mem-Ctrl, which achieved complete timing closure after global routing (WNS = 0, VEP = 0), and WB-DMA, which recovered +59.6% WNS and +63.2% TNS over the post-CTS baseline. Across all seven designs, our implementation consistently leaves fewer violating endpoints and better timing than the Original RMP after global routing, confirming that the physically-aware oracle produces cleaner restructurings that hold up under real routing parasitics.

5.1 Future Works in Consideration

- The cone extraction logic is an area that still needs more study. The current approach takes the full fan-in cone of the selected failing endpoints, which works reasonably well on the benchmarks tested but may not generalise to all designs. For example, on a heavily connected design, even selecting a small number of endpoints can pull in most of the logic, resulting in a cone that is too large to restructure effectively. A smarter extraction strategy that limits the cone to the most timing-critical portion of the fan-in, rather than collecting everything reachable, could produce more focused and reliable improvements.

- The current framework uses a tiered evaluation strategy where incremental DPL and Steiner RC are computed per MCTS iteration, and a full `repair_setup` is reserved for the top-5 final candidates. An extension would be to run `repair_setup` inside every evaluation call. During testing, some moderately good restructurings improved significantly after gate sizing was adjusted to match the new structure, suggesting that certain candidates are undervalued by the kCheap evaluator. The tradeoff is a significant increase in runtime per iteration; a smarter scheduling policy that applies the full evaluator only when the MCTS tree has converged on a promising subtree could recover this cost.

- The current evaluation accounts for placement and wire RC but not routing congestion. A natural extension is to incorporate routing congestion awareness directly into the MCTS evaluation — for example, by estimating routing demand after each candidate insertion and adding a congestion penalty to the composite score. This would allow the search to avoid restructurings that improve timing locally but create hotspots in the routing grid.

References

[1] Tutu Ajayi, David Blaauw, Tse-Bao Chan, Chung-Kuan Cheng, Vidya A. Chhabria, Dong Kwan Choo, Matteo Coltella, Ronald Dreslinski, Mateus Fogaça, Soheil Hashemi, et al. Openroad: Toward a self-driving, open-source digital layout implementation tool chain, 2019.

[2] Andrew B. Kahng, Jens Lienig, Igor L. Markov, and Jin Hu. *VLSI Physical Design: From Graph Partitioning to Timing Closure*. Springer, Dordrecht, Netherlands, 2011.

[3] Mingxiao He, Pengcheng Huang, Zhenyu Zhao, and Peiyun Bian. CPA-remap: Critical-path-based physically aware remapping framework for timing optimization. In *2025 IEEE 43rd International Conference on Computer Design (ICCD)*, pages 393–400. IEEE, 2025.

[4] The OpenROAD Project. Openroad restructure (rmp) module tool documentation. `https://openroad.readthedocs.io/en/latest/main/src/rmp/README.html`, 2026. Accessed: June 2, 2026.

[5] Dell Technologies. Powerscale: Best practices for semiconductor EDA design environments. Technical white paper, Dell Technologies Info Hub, 2026. Accessed: June 2, 2026.

[6] Yi-Chen Lu, Siddhartha Nath, Vishal Khandelwal, and SungKyu Lim. Rl-sizer: Vlsi gate sizing for timing optimization using deep reinforcement learning. In *Proceedings*

of the 58th ACM/IEEE Design Automation Conference (DAC), 2021.

[7] Laung-Terng Wang, Yao-Wen Chang, and Kwang-Ting Cheng. *Electronic Design Automation: Synthesis, Verification, and Test*. Morgan Kaufmann, Burlington, MA, USA, 2009.

[8] Parallax Software and The OpenROAD Project. OpenSTA: A gate level static timing verifier. `https://github.com/The-OpenROAD-Project/OpenSTA`, 2026.

[9] The OpenROAD Project. OpenRCX: Open-source parasitic extraction. `https://github.com/The-OpenROAD-Project/OpenROAD/tree/master/src/rcx`, 2026.

[10] Antmicro. Optimizing the resynthesis process in OpenROAD with simulated annealing. `https://antmicro.com/blog/2025/11/simulated-annealing-based-resynthesis-in-openroad`, 2025.

[11] A. Hosny, S. Hashemi, M. Shalan, and S. Reda. DRiLLS: Deep reinforcement learning for logic synthesis. In *Proc. ASP-DAC*, pages 581–586, 2020.

[12] Z. Pei, F. Liu, Z. He, G. Chen, H. Zheng, K. Zhu, and B. Yu. AlphaSyn: Logic synthesis optimization with efficient Monte Carlo Tree Search. In *Proc. ICCAD*, 2023.

[13] Levente Kocsis and Csaba Szepesvári. Bandit based Monte-Carlo planning. In *Proc. European Conference on Machine Learning (ECML)*, pages 282–293, 2006.

[14] OpenCores Community. OpenCores open source hardware intellectual property. `https://opencores.org`, 2026.

[15] Fulvio Corno, Maurizio Rebaudengo, and Giovanni Squillero. Rt-level ITC'99 benchmarks and first atpg results. *IEEE Design & Test of Computers*, 17(3):44–53, 2000.